Dự án eShop – Shopping Cart với Spring Boot, JPA, Thymeleaf, Spring Security & JWT 2
0. Chuẩn bị môi trường 4
1. Tạo cơ sở dữ liệu bằng eShop.sql 4
2. Tạo project Maven trong IntelliJ 6
PHẦN 1 – CRUD với Thymeleaf 11
Bước 1.1 – Entity 11
Bước 1.2 – Exception và Repository 12
Bước 1.3 – Service 13
Bước 1.4 – Layout dùng chung (Thymeleaf fragment) 15
Bước 1.5 – CRUD Danh mục 17
Bước 1.6 – CRUD Sản phẩm 19
Bước 1.7 – Trang cửa hàng: xem, tìm kiếm, lọc, phân trang 23
Bước 1.8 – Chạy và kiểm thử Phần 1 25
PHẦN 2 – Giỏ hàng và Thanh toán 26
Bước 2.1 – Giỏ hàng trong Session 26
Bước 2.2 – Entity đơn hàng 31
Bước 2.3 – Form thanh toán và OrderService 33
Bước 2.4 – Controller thanh toán và đơn hàng 35
Bước 2.5 – Template thanh toán và đơn hàng 38
Bước 2.6 – Kiểm thử Phần 2 42
PHẦN 3 – Spring Security + JWT + 3 vai trò 42
3.1. Lý thuyết nền tảng (đọc kỹ trước khi code) 42
Bước 3.2 – Thêm dependency 53
Bước 3.3 – Entity User, Repository 54
Bước 3.4 – UserDetailsService: nối Spring Security với bảng `users` 56
Bước 3.5 – JwtService: tạo và kiểm tra token 56
Bước 3.6 – JwtAuthenticationFilter 57
Bước 3.7 – SecurityConfig (trái tim của Phần 3) 59
Bước 3.8 – Đăng nhập, đăng ký, đăng xuất 61
Bước 3.9 – Navbar theo vai trò (`sec:authorize`) 67
Bước 3.10 – Gắn đơn hàng với khách hàng đăng nhập 68
Bước 3.11 – SELLER: CRUD bảng `users` 71
Bước 3.12 – Lớp bảo vệ thứ hai: Method Security 75
Bước 3.13 – REST API với JWT Bearer 75
Kiểm thử tổng hợp 78
Lỗi thường gặp 80
Bài tập mở rộng 81

Dự án eShop – Shopping Cart với Spring Boot, JPA, Thymeleaf, Spring Security & JWT

Môi trường: Java · Spring Boot · Maven · IntelliJ IDEA Ultimate · MariaDB · Spring Data JPA

Nội dung:

Phần	Chức năng	Kỹ thuật chính
1	CRUD Danh mục (`category`) và Sản phẩm (`product`, có `image_url`), trang cửa hàng xem/tìm kiếm/phân trang	Spring MVC, Thymeleaf, Spring Data JPA, Bean Validation
2	Giỏ hàng (thêm, sửa số lượng, xoá), thanh toán (COD / chuyển khoản giả lập), lưu đơn hàng, trừ tồn kho	`@SessionScope`, `@Transactional`, quan hệ `@OneToMany`
3	Đăng ký, đăng nhập, phân quyền 3 vai trò, bảng `users` trong MariaDB, **JWT** cho web (cookie HttpOnly) và REST API (Bearer token)	Spring Security 6, `UserDetailsService`, BCrypt, `OncePerRequestFilter`, jwt

Ma trận phân quyền (Phần 3):

Chức năng	GUEST (khách – chưa đăng nhập)	CUSTOMER (khách hàng)	SELLER (người bán)
Xem, tìm kiếm sản phẩm `/`, `/products/**`	✅	✅	✅
Đăng ký tài khoản `/register`	✅	❌	❌
Đăng nhập `/login`	✅	–	–
Giỏ hàng, thanh toán, đơn hàng của tôi `/cart/**`, `/checkout`, `/orders/**`	❌ (→ chuyển tới login)	✅	❌
CRUD danh mục, sản phẩm, đơn hàng, tài `/admin/**`	❌	❌ (403)	✅
`GET /api/products/**`	✅	✅	✅
API: `POST/PUT/DELETE /api/products/**`	❌ 401	❌ 403	✅

GUEST là người dùng *chưa đăng nhập*, không có dòng trong bảng `users`. Spring Security gán cho họ một `AnonymousAuthenticationToken`; ta cấu hình để token này mang quyền `ROLE_GUEST`, nhờ vậy cả 3 vai trò đều viết được bằng `hasRole(...)`.

0. Chuẩn bị môi trường

0.1. Vì sao chọn JDK 17 + Spring Boot 3.2.x?

Spring Boot 3.2 yêu cầu tối thiểu JDK 17 và Maven 3.6.3:

Thành phần	Lựa chọn trong tài liệu
JDK 17 (LTS) – Spring Boot 3.2 yêu cầu Java 17 trở lên	✅ → **dùng JDK 17**
Maven 3.9.x	✅ → **dùng Maven 3.9.x**
Spring Boot **3.2.x** (Tomcat 10.1.x)	✅ → **dùng Spring Boot 3.2.12**

Spring Boot 3.2.12 đi kèm Spring Framework 6.1, **Spring Security 6.2**, Hibernate 6.4, Thymeleaf 3.1

Nếu muốn dùng bản mới hơn (ví dụ JDK 21 + Spring Boot 3.5.x), chỉ cần đổi `<java.version>` và phiên bản của `spring-boot-starter-parent` trong `pom.xml`; code trong tài liệu gần như giữ nguyên.

0.2. Cài đặt

1. **JDK 17** (Temurin hoặc Oracle). Kiểm tra: `java -version` → `17.x`.
2. **IntelliJ IDEA Ultimate Maven project**. Kiểm tra plugin **Lombok** đã bật; vào *Settings* → *Build, Execution, Deployment* → *Compiler* → *Annotation Processors* → tick **Enable annotation processing**.
3. **MariaDB Server 10.6+** (Windows: cài kèm HeidiSQL). Ghi nhớ mật khẩu `root`.

1. Tạo cơ sở dữ liệu bằng eShop.sql

1.1. Sơ đồ ERD

```
CATEGORY ||--o{ PRODUCT : "chứa"
PRODUCT  ||--o{ ORDER_DETAIL : "được mua trong"
ORDERS   ||--|{ ORDER_DETAIL : "gồm"
USERS    |o--o{ ORDERS : "đặt"

CATEGORY {
    BIGINT id PK
    VARCHAR name UK
    VARCHAR description
}
PRODUCT {
    BIGINT id PK
    VARCHAR name
    VARCHAR description
    DECIMAL price
    INT quantity "tồn kho"
```

```
        VARCHAR image_url
        BIGINT category_id FK
        DATETIME created_at
    }
    USERS {
        BIGINT id PK
        VARCHAR username UK
        VARCHAR password "BCrypt"
        VARCHAR full_name
        VARCHAR email UK
        VARCHAR phone
        VARCHAR address
        VARCHAR role "ROLE_SELLER | ROLE_CUSTOMER"
        BOOLEAN enabled
    }
    ORDERS {
        BIGINT id PK
        DATETIME order_date
        VARCHAR customer_name
        VARCHAR phone
        VARCHAR address
        VARCHAR payment_method "COD | BANKING"
        VARCHAR status "NEW | PAID | SHIPPING | DONE | CANCELLED"
        DECIMAL total_amount
        BIGINT user_id FK "NULL được"
    }
    ORDER_DETAIL {
        BIGINT id PK
        BIGINT order_id FK
        BIGINT product_id FK
        INT quantity
        DECIMAL unit_price "giá lúc mua"
    }
```

Bảng	Dùng ở	Ghi chú thiết kế
`category`	Phần 1	15 dòng mẫu; `name` UNIQUE
`product`	Phần 1	50 dòng mẫu; `price DECIMAL(15,2)`; `quantity` là tồn kho; `image_url` là ảnh minh hoạ
`orders`, `order_detail`	Phần 2	`unit_price` lưu giá *tại thời điểm mua* (giá sản phẩm có thể đổi sau này); `user_id` cho phép NULL để Phần 2 chạy được khi chưa có đăng nhập

Bảng	Dùng ở	Ghi chú thiết kế
`users`	Phần 3	Mỗi tài khoản 1 vai trò, lưu kèm tiền tố `ROLE_`; `enabled = 0` là bị khoá

1.2. Chạy script

1. Chép file **`eShop.sql`** (đi kèm tài liệu này) vào thư mục `database/` của project.
2. IntelliJ Ultimate: tab **Database** → **+** → **Data Source** → **MariaDB** → host `localhost`, port `3306`, user `root`, mật khẩu của bạn → (lần đầu) *Download missing driver files* → **Test Connection** → OK.
3. Mở `eShop.sql` → chọn data source vừa tạo ở góc trên → **Execute** (Ctrl+Enter, chọn *Run all*). Hoặc trong HeidiSQL: *File* → *Run SQL file*.
4. Câu `SELECT` cuối script phải trả về: `category 15`, `product 50`, `users 5`.

Tài khoản mẫu (mật khẩu đều là `123456`):

username	Vai trò	Ghi chú
`seller1`, `seller2`	ROLE_SELLER	Quản trị toàn bộ
`customer1`, `customer2`	ROLE_CUSTOMER	Mua hàng
`locked`	ROLE_CUSTOMER	`enabled = 0` → không đăng nhập được

> Ảnh sản phẩm lưu trong thư mục project `src/main/resources/static/images/products/` và đổi `image_url` thành `/images/products/ten-anh.jpg`.

2. Tạo project Maven trong IntelliJ

2.1. Tạo project

Spring Initializr trong IntelliJ chỉ còn cho chọn các bản Spring Boot mới (không còn 3.2.x), nên ta tạo **project Maven trống** rồi tự viết `pom.xml`:

1. *File* → *New* → *Project* → chọn generator **Java** (mục *New Project*).
2. Điền:

Mục	Giá trị
Name	`eshop`
Build system	**Maven**
JDK	**17**

Mục	Giá trị
Add sample code	bỏ tick
Advanced Settings → GroupId	`com.example`
Advanced Settings → ArtifactId	`eshop`

3. Bấm **Create**.

4. Kiểm tra lại cấu hình Maven: *Settings* → *Build, Execution, Deployment* → *Build Tools* → *Maven*:

   - **Maven** *(đã setup)*
   - **JDK for importer** (mục *Importing*): JDK **17**
   - **Local repository**: `%USERPROFILE%\.m2\repository` (mặc định)

Chạy từ terminal: `mvn spring-boot:run`.

2.2. Cấu trúc thư mục chuẩn của Maven

Maven đặt tên project ngay trong `pom.xml` (thẻ `<artifactId>`) và dùng cấu trúc thư mục chuẩn sau:

```text
pom.xml              # khai báo project, thư viện, plugin build
src/main/java        # mã nguồn Java
src/main/resources   # application.properties, templates/, static/
src/test/java        # mã nguồn test
```

2.3. `pom.xml` (Phần 1 & 2)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <!-- Spring Boot quản lý sẵn phiên bản thư viện -->
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.12</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>eshop</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>eshop</name>
```

```
    <description>eShop - Shopping Cart</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <dependency>
            <groupId>org.mariadb.jdbc</groupId>
            <artifactId>mariadb-java-client</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
```

```
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Load Maven Changes (chữ *m* có vòng xoay) ở góc phải editor, hoặc tab *Maven* → *Reload All Maven Projects*.

Chờ tải xong thư viện, Kiểm tra bằng terminal: `mvn clean compile` phải báo `BUILD SUCCESS`.

Phần 3 sẽ bổ sung Spring Security và jwt. **Chưa thêm security ở Phần 1–2**, vì vừa thêm starter security là mọi URL bị khoá ngay.

2.4. Lớp khởi động

`src/main/java/com/example/eshop/EshopApplication.java`

```
package com.example.eshop;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EshopApplication {
    public static void main(String[] args) {
        SpringApplication.run(EshopApplication.class, args);
    }
}
```

2.5. `src/main/resources/application.properties`

```
spring.application.name=eshop
server.port=8080

# ===== MariaDB =====
spring.datasource.url=jdbc:mariadb://localhost:3306/eshop
spring.datasource.username=root
# Doi mat khau theo may cua ban
spring.datasource.password=……
```

```
# ===== JPA / Hibernate =====
# Bang da duoc tao bang eShop.sql nen khong cho Hibernate tao/sua bang
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
# Cho phep Thymeleaf doc quan he LAZY (vd: product.category.name)
spring.jpa.open-in-view=true

# ===== Thymeleaf =====
spring.thymeleaf.cache=false
```

Lưu ý: file `.properties` **không** cho phép chú thích cùng dòng với giá trị.

2.6. Cấu trúc project hoàn chỉnh (sau cả 3 phần)

```
eshop/
├── pom.xml
├── database/eShop.sql
└── src/main/
    ├── java/com/example/eshop/
    │   ├── EshopApplication.java
    │   ├── entity/        Category, Product, Order, OrderDetail, OrderStatus, PaymentMethod,
User, Role
    │   ├── repository/    CategoryRepository, ProductRepository, OrderRepository, UserRepository
    │   ├── exception/     NotFoundException
    │   ├── dto/           CheckoutForm, RegisterForm, UserForm
    │   ├── cart/          Cart, CartItem
    │   ├── service/       CategoryService, ProductService, OrderService, UserService
    │   ├── security/      SecurityConfig, JwtService, JwtAuthenticationFilter,
CustomUserDetailsService   (Phần 3)
    │   ├── controller/    ShopController, CartController, CheckoutController, OrderController,
    │   │                  AuthController, ErrorPageController, GlobalModelAttributes
    │   ├── controller/admin/  AdminCategoryController, AdminProductController,
    │   │                      AdminOrderController, AdminUserController
    │   └── api/           AuthApiController, ProductApiController, ApiExceptionHandler   (Phần 3)
    └── resources/
        ├── application.properties
        ├── static/css/style.css
        └── templates/
            ├── fragments/layout.html
            ├── shop/       index.html, detail.html
            ├── admin/category/  list.html, form.html
            ├── admin/product/   list.html, form.html
            ├── admin/order/     list.html, detail.html
            ├── admin/user/      list.html, form.html        (Phần 3)
            ├── cart/view.html
            ├── checkout/form.html
            ├── order/      payment.html, detail.html, my-orders.html
            ├── auth/       login.html, register.html       (Phần 3)
            └── error/403.html                              (Phần 3)
```

PHẦN 1 – CRUD với Thymeleaf

Mục tiêu: người bán quản lý danh mục và sản phẩm tại `/admin/...`; người mua xem cửa hàng tại `/`, tìm theo tên, lọc theo danh mục, phân trang, xem chi tiết.

Bước 1.1 – Entity

entity/Category.java

```
package com.example.eshop.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Table(name = "category")
@Getter @Setter @NoArgsConstructor
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Tên danh mục không được để trống")
    @Size(max = 100, message = "Tối đa 100 ký tự")
    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @Size(max = 255, message = "Tối đa 255 ký tự")
    private String description;
}
```

entity/Product.java

```
package com.example.eshop.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import org.hibernate.annotations.CreationTimestamp;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "product")
@Getter @Setter @NoArgsConstructor
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    @NotBlank(message = "Tên sản phẩm không được để trống")
    @Size(max = 150, message = "Tối đa 150 ký tự")
    @Column(nullable = false, length = 150)
    private String name;

    @Size(max = 500, message = "Tối đa 500 ký tự")
    private String description;

    @NotNull(message = "Vui lòng nhập giá")
    @DecimalMin(value = "0", message = "Giá phải >= 0")
    @Column(nullable = false, precision = 15, scale = 2)
    private BigDecimal price;

    @NotNull(message = "Vui lòng nhập số lượng")
    @Min(value = 0, message = "Số lượng phải >= 0")
    @Column(nullable = false)
    private Integer quantity;

    @Size(max = 500)
    @Column(name = "image_url", length = 500)
    private String imageUrl;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id", nullable = false)
    private Category category;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;
}
```

Giải thích:

- `@ManyToOne(fetch = LAZY)`: nhiều sản phẩm thuộc 1 danh mục; chỉ nạp danh mục khi cần.
- Tên thuộc tính `imageUrl` ↔ cột `image_url`: Spring Boot tự đổi camelCase → snake_case, `@Column(name=...)` chỉ để rõ ràng.
- `updatable = false` cho `createdAt`: khi sửa sản phẩm, form không gửi `createdAt` nhưng cột này không bị ghi đè thành NULL.
- Dùng `@Getter @Setter` thay cho `@Data` với entity có quan hệ để tránh `toString()/hashCode()` lặp vô hạn.

Bước 1.2 – Exception và Repository

`exception/NotFoundException.java`

```
package com.example.eshop.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) { super(message); }
```

```
}
```

repository/CategoryRepository.java

```
package com.example.eshop.repository;

import com.example.eshop.entity.Category;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface CategoryRepository extends JpaRepository<Category, Long> {
    List<Category> findAllByOrderByNameAsc();
}
```

repository/ProductRepository.java

```
package com.example.eshop.repository;

import com.example.eshop.entity.Product;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface ProductRepository extends JpaRepository<Product, Long> {

    boolean existsByCategoryId(Long categoryId);

    /** Tìm theo tên (không phân biệt hoa thường) và/hoặc theo danh mục; tham số null = bỏ qua điều
kiện. */
    @Query("""
           SELECT p FROM Product p
           WHERE (:keyword IS NULL OR LOWER(p.name) LIKE LOWER(CONCAT('%', :keyword, '%')))
             AND (:categoryId IS NULL OR p.category.id = :categoryId)
           """)
    Page<Product> search(@Param("keyword") String keyword,
                         @Param("categoryId") Long categoryId,
                         Pageable pageable);
}
```

Bước 1.3 – Service

service/CategoryService.java

```
package com.example.eshop.service;

import com.example.eshop.entity.Category;
import com.example.eshop.exception.NotFoundException;
import com.example.eshop.repository.CategoryRepository;
import com.example.eshop.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;
```

```
@Service
@RequiredArgsConstructor
public class CategoryService {

    private final CategoryRepository categoryRepository;
    private final ProductRepository productRepository;

    public List<Category> findAll() {
        return categoryRepository.findAllByOrderByNameAsc();
    }

    public Category findById(Long id) {
        return categoryRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("Không tìm thấy danh mục id = " + id));
    }

    public Category save(Category category) {
        return categoryRepository.save(category);
    }

    public void delete(Long id) {
        if (productRepository.existsByCategoryId(id)) {
            throw new IllegalStateException("Danh mục đang có sản phẩm, không thể xoá");
        }
        categoryRepository.deleteById(id);
    }
}
```

service/ProductService.java

```
package com.example.eshop.service;

import com.example.eshop.entity.Category;
import com.example.eshop.entity.Product;
import com.example.eshop.exception.NotFoundException;
import com.example.eshop.repository.CategoryRepository;
import com.example.eshop.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.util.StringUtils;

@Service
@RequiredArgsConstructor
public class ProductService {

    private final ProductRepository productRepository;
    private final CategoryRepository categoryRepository;

    public Page<Product> search(String keyword, Long categoryId, int page, int size) {
        String kw = StringUtils.hasText(keyword) ? keyword.trim() : null;
        return productRepository.search(kw, categoryId,
                PageRequest.of(Math.max(page, 0), size, Sort.by("id").descending()));
    }
```

```
    public Product findById(Long id) {
        return productRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("Không tìm thấy sản phẩm id = " + id));
    }

    @Transactional
    public Product save(Product product, Long categoryId) {
        Category category = categoryRepository.findById(categoryId)
                .orElseThrow(() -> new NotFoundException("Không tìm thấy danh mục id = " +
categoryId));
        product.setCategory(category);
        return productRepository.save(product);   // id == null → INSERT, id != null → UPDATE
    }

    public void delete(Long id) {
        try {
            productRepository.deleteById(id);
        } catch (DataIntegrityViolationException e) {
            // vướng khoá ngoại order_detail.product_id
            throw new IllegalStateException("Sản phẩm đã có trong đơn hàng, không thể xoá");
        }
    }
}
```

Bước 1.4 – Layout dùng chung (Thymeleaf fragment)

`templates/fragments/layout.html` – **phiên bản Phần 1** (Phần 2 và 3 sẽ bổ sung menu)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head th:fragment="head(title)">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title th:text="${title} + ' | eShop'">eShop</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link th:href="@{/css/style.css}" rel="stylesheet">
</head>

<body>

<nav th:fragment="navbar" class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
    <div class="container">
        <a class="navbar-brand fw-bold" th:href="@{/}">🛒 eShop</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#mainNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="mainNav">
            <ul class="navbar-nav me-auto">
                <li class="nav-item"><a class="nav-link" th:href="@{/products}">Sản phẩm</a></li>
                <li class="nav-item dropdown">
                    <a class="nav-link dropdown-toggle" href="#" data-bs-toggle="dropdown">Quản
trị</a>
                    <ul class="dropdown-menu">
                        <li><a class="dropdown-item" th:href="@{/admin/categories}">Danh
mục</a></li>
```

```
                        <li><a class="dropdown-item" th:href="@{/admin/products}">Sản phẩm</a></li>
                    </ul>
                </li>
            </ul>
            <form class="d-flex" th:action="@{/products}" method="get">
                <input class="form-control me-2" name="keyword" placeholder="Tìm sản phẩm..."
th:value="${keyword}">
                <button class="btn btn-light" type="submit">Tìm</button>
            </form>
        </div>
    </div>
</nav>

<div th:fragment="alerts">
    <div th:if="${message}" class="alert alert-success alert-dismissible fade show">
        <span th:text="${message}"></span>
        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
    <div th:if="${error}" class="alert alert-danger alert-dismissible fade show">
        <span th:text="${error}"></span>
        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
</div>

<footer th:fragment="footer" class="border-top mt-5 py-3 text-center text-muted small">
    eShop — Demo Spring Boot · Thymeleaf · JPA · MariaDB
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</footer>

</body>
</html>
```

static/css/style.css

```
.product-img   { height: 180px; object-fit: cover; }
.thumb         { width: 64px; height: 48px; object-fit: cover; }
.card-title    { min-height: 2.6em; }
.price         { color: #d70018; font-weight: 700; }
```

Khung chung của mọi trang (các template bên dưới đều theo mẫu này):

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Tiêu đề trang')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>
    <!-- nội dung trang -->
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

Bước 1.5 – CRUD Danh mục

`controller/admin/AdminCategoryController.java`

```
package com.example.eshop.controller.admin;

import com.example.eshop.entity.Category;
import com.example.eshop.service.CategoryService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/admin/categories")
@RequiredArgsConstructor
public class AdminCategoryController {

    private final CategoryService categoryService;

    @GetMapping
    public String list(Model model) {
        model.addAttribute("categories", categoryService.findAll());
        return "admin/category/list";
    }

    @GetMapping("/new")
    public String createForm(Model model) {
        model.addAttribute("category", new Category());
        return "admin/category/form";
    }

    @GetMapping("/{id}/edit")
    public String editForm(@PathVariable Long id, Model model) {
        model.addAttribute("category", categoryService.findById(id));
        return "admin/category/form";
    }

    @PostMapping("/save")
    public String save(@Valid @ModelAttribute("category") Category category,
                       BindingResult result, RedirectAttributes ra) {
        if (result.hasErrors()) {
            return "admin/category/form";
        }
        try {
            categoryService.save(category);
        } catch (DataIntegrityViolationException e) {
            result.rejectValue("name", "duplicate", "Tên danh mục đã tồn tại");
            return "admin/category/form";
        }
        ra.addFlashAttribute("message", "Đã lưu danh mục \"" + category.getName() + "\"");
        return "redirect:/admin/categories";
    }

    @PostMapping("/{id}/delete")
    public String delete(@PathVariable Long id, RedirectAttributes ra) {
```

```
        try {
            categoryService.delete(id);
            ra.addFlashAttribute("message", "Đã xoá danh mục");
        } catch (IllegalStateException e) {
            ra.addFlashAttribute("error", e.getMessage());
        }
        return "redirect:/admin/categories";
    }
}
```

Mẫu PRG (Post – Redirect – Get): sau khi POST thành công luôn `redirect:` để F5 không gửi lại form; thông báo chuyển qua `RedirectAttributes.addFlashAttribute` (sống qua đúng 1 lần redirect).

templates/admin/category/list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Danh mục')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>

    <div class="d-flex justify-content-between align-items-center mb-3">
        <h3>Danh mục sản phẩm</h3>
        <a class="btn btn-success" th:href="@{/admin/categories/new}">+ Thêm danh mục</a>
    </div>

    <table class="table table-bordered table-hover align-middle">
        <thead class="table-light">
        <tr><th style="width:70px">ID</th><th>Tên</th><th>Mô tả</th><th
style="width:170px"></th></tr>
        </thead>
        <tbody>
        <tr th:each="c : ${categories}">
            <td th:text="${c.id}"></td>
            <td th:text="${c.name}"></td>
            <td th:text="${c.description}"></td>
            <td class="text-nowrap">
                <a class="btn btn-sm btn-outline-primary"
th:href="@{/admin/categories/{id}/edit(id=${c.id})}">Sửa</a>
                <form th:action="@{/admin/categories/{id}/delete(id=${c.id})}" method="post"
class="d-inline"
                      onsubmit="return confirm('Xoá danh mục này?')">
                    <button class="btn btn-sm btn-outline-danger">Xoá</button>
                </form>
            </td>
        </tr>
        <tr th:if="${#lists.isEmpty(categories)}"><td colspan="4" class="text-center">Chưa có dữ
liệu</td></tr>
        </tbody>
    </table>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

templates/admin/category/form.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Danh mục')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container" style="max-width:640px">
    <h3 th:text="${category.id == null} ? 'Thêm danh mục' : 'Sửa danh mục'"></h3>

    <form th:action="@{/admin/categories/save}" th:object="${category}" method="post">
        <input type="hidden" th:field="*{id}">
        <div class="mb-3">
            <label class="form-label">Tên danh mục *</label>
            <input class="form-control" th:field="*{name}" th:errorclass="is-invalid">
            <div class="invalid-feedback" th:errors="*{name}"></div>
        </div>
        <div class="mb-3">
            <label class="form-label">Mô tả</label>
            <textarea class="form-control" rows="3" th:field="*{description}" th:errorclass="is-
invalid"></textarea>
            <div class="invalid-feedback" th:errors="*{description}"></div>
        </div>
        <button class="btn btn-primary">Lưu</button>
        <a class="btn btn-secondary" th:href="@{/admin/categories}">Huỷ</a>
    </form>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

Bước 1.6 – CRUD Sản phẩm

controller/admin/AdminProductController.java

```
package com.example.eshop.controller.admin;

import com.example.eshop.entity.Product;
import com.example.eshop.service.CategoryService;
import com.example.eshop.service.ProductService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/admin/products")
@RequiredArgsConstructor
public class AdminProductController {

    private final ProductService productService;
    private final CategoryService categoryService;

    @GetMapping
    public String list(@RequestParam(required = false) String keyword,
```

```
                    @RequestParam(required = false) Long categoryId,
                    @RequestParam(defaultValue = "0") int page,
                    Model model) {
        model.addAttribute("products", productService.search(keyword, categoryId, page, 10));
        model.addAttribute("categories", categoryService.findAll());
        model.addAttribute("keyword", keyword);
        model.addAttribute("categoryId", categoryId);
        return "admin/product/list";
    }

    @GetMapping("/new")
    public String createForm(Model model) {
        Product p = new Product();
        p.setQuantity(0);
        return showForm(model, p, null);
    }

    @GetMapping("/{id}/edit")
    public String editForm(@PathVariable Long id, Model model) {
        Product p = productService.findById(id);
        return showForm(model, p, p.getCategory().getId());
    }

    @PostMapping("/save")
    public String save(@Valid @ModelAttribute("product") Product product,
                    BindingResult result,
                    @RequestParam(required = false) Long categoryId,
                    Model model, RedirectAttributes ra) {
        if (categoryId == null) {
            model.addAttribute("categoryError", "Vui lòng chọn danh mục");
        }
        if (result.hasErrors() || categoryId == null) {
            return showForm(model, product, categoryId);
        }
        productService.save(product, categoryId);
        ra.addFlashAttribute("message", "Đã lưu sản phẩm \"" + product.getName() + "\"");
        return "redirect:/admin/products";
    }

    @PostMapping("/{id}/delete")
    public String delete(@PathVariable Long id, RedirectAttributes ra) {
        try {
            productService.delete(id);
            ra.addFlashAttribute("message", "Đã xoá sản phẩm");
        } catch (IllegalStateException e) {
            ra.addFlashAttribute("error", e.getMessage());
        }
        return "redirect:/admin/products";
    }

    private String showForm(Model model, Product product, Long selectedCategoryId) {
        model.addAttribute("product", product);
        model.addAttribute("categories", categoryService.findAll());
        model.addAttribute("selectedCategoryId", selectedCategoryId);
        return "admin/product/form";
    }
}
```

Danh mục được gửi lên bằng tham số riêng `categoryId` (thẻ `<select name="categoryId">`), service sẽ

nạp `Category` từ DB rồi gán cho sản phẩm. Cách này đơn giản hơn binding lồng `category.id`.

templates/admin/product/list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Quản lý sản phẩm')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>

    <div class="d-flex justify-content-between align-items-center mb-3">
        <h3>Quản lý sản phẩm</h3>
        <a class="btn btn-success" th:href="@{/admin/products/new}">+ Thêm sản phẩm</a>
    </div>

    <form class="row g-2 mb-3" th:action="@{/admin/products}" method="get">
        <div class="col-md-5"><input class="form-control" name="keyword" th:value="${keyword}"
placeholder="Tên sản phẩm"></div>
        <div class="col-md-4">
            <select class="form-select" name="categoryId">
                <option value="">-- Tất cả danh mục --</option>
                <option th:each="c : ${categories}" th:value="${c.id}" th:text="${c.name}"
                        th:selected="${c.id == categoryId}"></option>
            </select>
        </div>
        <div class="col-md-3"><button class="btn btn-primary w-100">Lọc</button></div>
    </form>

    <table class="table table-bordered table-hover align-middle">
        <thead class="table-light">
        <tr><th>ID</th><th>Ảnh</th><th>Tên</th><th>Danh mục</th><th class="text-end">Giá</th><th
class="text-end">Tồn</th><th></th></tr>
        </thead>
        <tbody>
        <tr th:each="p : ${products.content}">
            <td th:text="${p.id}"></td>
            <td><img class="thumb rounded" th:src="${p.imageUrl}" alt=""></td>
            <td th:text="${p.name}"></td>
            <td th:text="${p.category.name}"></td>
            <td class="text-end" th:text="${#numbers.formatDecimal(p.price, 0, 'POINT', 0,
'COMMA')} + ' ₫'"></td>
            <td class="text-end" th:text="${p.quantity}"></td>
            <td class="text-nowrap">
                <a class="btn btn-sm btn-outline-primary"
th:href="@{/admin/products/{id}/edit(id=${p.id})}">Sửa</a>
                <form th:action="@{/admin/products/{id}/delete(id=${p.id})}" method="post"
class="d-inline"
                      onsubmit="return confirm('Xoá sản phẩm này?')">
                    <button class="btn btn-sm btn-outline-danger">Xoá</button>
                </form>
            </td>
        </tr>
        </tbody>
    </table>

    <nav th:if="${products.totalPages > 1}">
        <ul class="pagination justify-content-center">
            <li class="page-item" th:each="i : ${#numbers.sequence(0, products.totalPages - 1)}"
```

```
                th:classappend="${i == products.number} ? 'active'">
                <a class="page-link" th:href="@{/admin/products(keyword=${keyword},
categoryId=${categoryId}, page=${i})}"
                   th:text="${i + 1}"></a>
            </li>
        </ul>
    </nav>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

templates/admin/product/form.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Sản phẩm')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container" style="max-width:760px">
    <h3 th:text="${product.id == null} ? 'Thêm sản phẩm' : 'Sửa sản phẩm'"></h3>

    <form th:action="@{/admin/products/save}" th:object="${product}" method="post">
        <input type="hidden" th:field="*{id}">

        <div class="mb-3">
            <label class="form-label">Tên sản phẩm *</label>
            <input class="form-control" th:field="*{name}" th:errorclass="is-invalid">
            <div class="invalid-feedback" th:errors="*{name}"></div>
        </div>

        <div class="mb-3">
            <label class="form-label">Danh mục *</label>
            <select class="form-select" name="categoryId" th:classappend="${categoryError} ? 'is-
invalid'">
                <option value="">-- Chọn danh mục --</option>
                <option th:each="c : ${categories}" th:value="${c.id}" th:text="${c.name}"
                        th:selected="${c.id == selectedCategoryId}"></option>
            </select>
            <div class="invalid-feedback" th:text="${categoryError}"></div>
        </div>

        <div class="row">
            <div class="col-md-6 mb-3">
                <label class="form-label">Giá (đ) *</label>
                <input type="number" step="1000" min="0" class="form-control" th:field="*{price}"
th:errorclass="is-invalid">
                <div class="invalid-feedback" th:errors="*{price}"></div>
            </div>
            <div class="col-md-6 mb-3">
                <label class="form-label">Số lượng tồn *</label>
                <input type="number" min="0" class="form-control" th:field="*{quantity}"
th:errorclass="is-invalid">
                <div class="invalid-feedback" th:errors="*{quantity}"></div>
            </div>
        </div>

        <div class="mb-3">
            <label class="form-label">URL hình ảnh</label>
```

```
            <input class="form-control" th:field="*{imageUrl}" placeholder="https://... hoặc
/images/products/a.jpg">
            <img th:if="*{imageUrl}" th:src="*{imageUrl}" class="mt-2 rounded border" style="max-
height:150px" alt="">
        </div>

        <div class="mb-3">
            <label class="form-label">Mô tả</label>
            <textarea class="form-control" rows="3" th:field="*{description}" th:errorclass="is-
invalid"></textarea>
            <div class="invalid-feedback" th:errors="*{description}"></div>
        </div>

        <button class="btn btn-primary">Lưu</button>
        <a class="btn btn-secondary" th:href="@{/admin/products}">Huỷ</a>
    </form>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

Bước 1.7 – Trang cửa hàng: xem, tìm kiếm, lọc, phân trang

`controller/ShopController.java`

```
package com.example.eshop.controller;

import com.example.eshop.service.CategoryService;
import com.example.eshop.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequiredArgsConstructor
public class ShopController {

    private final ProductService productService;
    private final CategoryService categoryService;

    @GetMapping({"/", "/products"})
    public String index(@RequestParam(required = false) String keyword,
                        @RequestParam(required = false) Long categoryId,
                        @RequestParam(defaultValue = "0") int page,
                        Model model) {
        model.addAttribute("products", productService.search(keyword, categoryId, page, 12));
        model.addAttribute("categories", categoryService.findAll());
        model.addAttribute("keyword", keyword);
        model.addAttribute("categoryId", categoryId);
        return "shop/index";
    }

    @GetMapping("/products/{id}")
    public String detail(@PathVariable Long id, Model model) {
        model.addAttribute("product", productService.findById(id));
        return "shop/detail";
    }
```

```
}
```

templates/shop/index.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Cửa hàng')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>
    <div class="row">
        <!-- Cột danh mục -->
        <aside class="col-md-3 mb-3">
            <div class="list-group">
                <a class="list-group-item list-group-item-action"
                   th:classappend="${categoryId == null} ? 'active'"
                   th:href="@{/products(keyword=${keyword})}">Tất cả</a>
                <a th:each="c : ${categories}" class="list-group-item list-group-item-action"
                   th:classappend="${c.id == categoryId} ? 'active'"
                   th:href="@{/products(categoryId=${c.id}, keyword=${keyword})}"
th:text="${c.name}"></a>
            </div>
        </aside>

        <!-- Lưới sản phẩm -->
        <section class="col-md-9">
            <p class="text-muted">
                Tìm thấy <b th:text="${products.totalElements}">0</b> sản phẩm
                <span th:if="${keyword}"> cho từ khoá "<b th:text="${keyword}"></b>"</span>
            </p>
            <div class="row row-cols-2 row-cols-lg-3 g-3">
                <div class="col" th:each="p : ${products.content}">
                    <div class="card h-100 shadow-sm">
                        <a th:href="@{/products/{id}(id=${p.id})}">
                            <img class="card-img-top product-img" th:src="${p.imageUrl}"
th:alt="${p.name}">
                        </a>
                        <div class="card-body d-flex flex-column">
                            <small class="text-muted" th:text="${p.category.name}"></small>
                            <h6 class="card-title" th:text="${p.name}"></h6>
                            <div class="price mb-2" th:text="${#numbers.formatDecimal(p.price, 0,
'POINT', 0, 'COMMA')} + ' ₫'"></div>
                            <div class="mt-auto">
                                <!-- Nút "Thêm vào giỏ" sẽ bổ sung ở Phần 2 -->
                                <a class="btn btn-sm btn-outline-primary"
th:href="@{/products/{id}(id=${p.id})}">Chi tiết</a>
                            </div>
                        </div>
                    </div>
                </div>
            </div>

            <nav class="mt-4" th:if="${products.totalPages > 1}">
                <ul class="pagination justify-content-center">
                    <li class="page-item" th:each="i : ${#numbers.sequence(0, products.totalPages -
1)}"
                        th:classappend="${i == products.number} ? 'active'">
                        <a class="page-link" th:href="@{/products(keyword=${keyword},
```

```
categoryId=${categoryId}, page=${i})}"
                            th:text="${i + 1}"></a>
                    </li>
                </ul>
            </nav>
        </section>
    </div>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

templates/shop/detail.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head(${product.name})}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>
    <div class="row g-4">
        <div class="col-md-5"><img class="img-fluid rounded border" th:src="${product.imageUrl}"
th:alt="${product.name}"></div>
        <div class="col-md-7">
            <small class="text-muted" th:text="${product.category.name}"></small>
            <h3 th:text="${product.name}"></h3>
            <h4 class="price" th:text="${#numbers.formatDecimal(product.price, 0, 'POINT', 0,
'COMMA')} + ' đ'"></h4>
            <p th:text="${product.description}"></p>
            <p>Còn lại: <b th:text="${product.quantity}"></b> sản phẩm</p>
            <!-- Form "Thêm vào giỏ" sẽ bổ sung ở Phần 2 -->
            <a class="btn btn-link px-0" th:href="@{/products}">← Tiếp tục xem hàng</a>
        </div>
    </div>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

Bước 1.8 – Chạy và kiểm thử Phần 1

Chạy `EshopApplication` (nút ▶️ cạnh `main`, hoặc tab *Maven* → *Plugins* → *spring-boot* → *spring-boot:run*, hoặc terminal: `mvn spring-boot:run`).

#	Thao tác	Kết quả mong đợi
1	Mở http://localhost:8080	12 sản phẩm/trang, 5 trang (50 sản phẩm), cột trái 15 danh mục
2	Bấm danh mục *Laptop*	Còn 4 sản phẩm
3	Gõ "samsung" vào ô tìm	Galaxy S24, Galaxy Tab S9 FE, Galaxy Watch6, T7, Odyssey G5

#	Thao tác	Kết quả mong đợi
4	`/admin/categories` → Thêm "Điện thoại"	Báo lỗi *Tên danh mục đã tồn tại*
5	Xoá danh mục *Laptop*	Báo lỗi *Danh mục đang có sản phẩm*
6	Thêm danh mục mới rồi xoá	Thành công
7	`/admin/products/new` → để trống tên, giá âm	Hiện lỗi đỏ dưới từng ô
8	Sửa giá một sản phẩm	Bảng `product` trong DB cập nhật, `created_at` giữ nguyên

PHẦN 2 – Giỏ hàng và Thanh toán

Luồng nghiệp vụ:

```
Xem sản phẩm → [Thêm vào giỏ] → Giỏ hàng (sửa SL / xoá) → [Thanh toán]
     → Nhập họ tên, SĐT, địa chỉ, chọn phương thức
           ├─ COD      → Lưu đơn (NEW) ──────────────────────→ Trang đơn hàng
           └─ BANKING  → Lưu đơn (NEW) → Trang chuyển khoản → [Xác nhận] → PAID → Trang đơn
hàng
Khi lưu đơn: kiểm tra tồn kho, trừ tồn kho, lưu order + order_detail trong 1 transaction, xoá giỏ
```

Giỏ hàng chưa thanh toán **không lưu DB** mà lưu trong `HttpSession` của từng người dùng (bean `@SessionScope`).

Bước 2.1 – Giỏ hàng trong Session

`cart/CartItem.java`

```
package com.example.eshop.cart;

import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.Setter;
import java.io.Serializable;
import java.math.BigDecimal;

@Getter
@AllArgsConstructor
public class CartItem implements Serializable {
    private Long productId;
    private String name;
    private String imageUrl;
    private BigDecimal price;
    @Setter
    private int quantity;

    public BigDecimal getSubtotal() {
        return price.multiply(BigDecimal.valueOf(quantity));
    }
}
```

cart/Cart.java

```
package com.example.eshop.cart;

import com.example.eshop.entity.Product;
import org.springframework.stereotype.Component;
import org.springframework.web.context.annotation.SessionScope;

import java.io.Serializable;
import java.math.BigDecimal;
import java.util.Collection;
import java.util.LinkedHashMap;
import java.util.Map;

/** Mỗi HttpSession (mỗi trình duyệt) có một đối tượng Cart riêng. */
@Component
@SessionScope
public class Cart implements Serializable {

    private final Map<Long, CartItem> items = new LinkedHashMap<>();

    public void add(Product p, int quantity) {
        CartItem item = items.get(p.getId());
        if (item == null) {
            items.put(p.getId(), new CartItem(p.getId(), p.getName(), p.getImageUrl(),
p.getPrice(), quantity));
        } else {
            item.setQuantity(item.getQuantity() + quantity);
        }
    }

    public void update(Long productId, int quantity) {
        if (quantity <= 0) {
            items.remove(productId);
        } else if (items.containsKey(productId)) {
            items.get(productId).setQuantity(quantity);
        }
    }

    public void remove(Long productId) { items.remove(productId); }

    public void clear() { items.clear(); }

    public Collection<CartItem> getItems() { return items.values(); }

    public boolean isEmpty() { return items.isEmpty(); }

    public int getQuantityOf(Long productId) {
        CartItem i = items.get(productId);
        return i == null ? 0 : i.getQuantity();
    }

    public int getTotalQuantity() {
        return items.values().stream().mapToInt(CartItem::getQuantity).sum();
    }

    public BigDecimal getTotalAmount() {
        return items.values().stream().map(CartItem::getSubtotal).reduce(BigDecimal.ZERO,
```

```
BigDecimal::add);
    }
}
```

`@SessionScope` tạo một **proxy**: controller (singleton) giữ proxy, mỗi lần gọi phương thức proxy sẽ tìm đúng đối tượng `Cart` trong session của request hiện tại. Đây là ví dụ hay về *bean scope* trong Spring.

`controller/GlobalModelAttributes.java` – đưa số lượng trong giỏ vào **mọi** trang (để hiện trên navbar):

```
package com.example.eshop.controller;

import com.example.eshop.cart.Cart;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ModelAttribute;

@ControllerAdvice(basePackages = "com.example.eshop.controller")
@RequiredArgsConstructor
public class GlobalModelAttributes {

    private final Cart cart;

    @ModelAttribute("cartCount")
    public int cartCount() {
        return cart.getTotalQuantity();
    }
}
```

`controller/CartController.java`

```
package com.example.eshop.controller;

import com.example.eshop.cart.Cart;
import com.example.eshop.entity.Product;
import com.example.eshop.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/cart")
@RequiredArgsConstructor
public class CartController {

    private final Cart cart;
    private final ProductService productService;

    @GetMapping
    public String view(Model model) {
        model.addAttribute("cart", cart);
        return "cart/view";
    }

    @PostMapping("/add")
    public String add(@RequestParam Long productId,
```

```
                    @RequestParam(defaultValue = "1") int quantity,
                    RedirectAttributes ra) {
        Product p = productService.findById(productId);
        int wanted = cart.getQuantityOf(productId) + quantity;
        if (quantity < 1 || wanted > p.getQuantity()) {
            ra.addFlashAttribute("error", "Không đủ hàng: \"" + p.getName() + "\" chỉ còn " +
p.getQuantity());
            return "redirect:/products/" + productId;
        }
        cart.add(p, quantity);
        ra.addFlashAttribute("message", "Đã thêm \"" + p.getName() + "\" vào giỏ hàng");
        return "redirect:/cart";
    }

    @PostMapping("/update")
    public String update(@RequestParam Long productId, @RequestParam int quantity,
RedirectAttributes ra) {
        Product p = productService.findById(productId);
        if (quantity > p.getQuantity()) {
            ra.addFlashAttribute("error", "\"" + p.getName() + "\" chỉ còn " + p.getQuantity());
        } else {
            cart.update(productId, quantity);
        }
        return "redirect:/cart";
    }

    @PostMapping("/remove/{productId}")
    public String remove(@PathVariable Long productId) {
        cart.remove(productId);
        return "redirect:/cart";
    }

    @PostMapping("/clear")
    public String clear() {
        cart.clear();
        return "redirect:/cart";
    }
}
```

templates/cart/view.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Giỏ hàng')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>
    <h3>Giỏ hàng</h3>

    <div th:if="${cart.empty}" class="alert alert-info">
        Giỏ hàng trống. <a th:href="@{/products}">Mua sắm ngay</a>
    </div>

    <div th:unless="${cart.empty}">
        <table class="table align-middle">
            <thead><tr><th>Sản phẩm</th><th class="text-end">Đơn giá</th><th style="width:170px">Số
lượng</th><th class="text-end">Thành tiền</th><th></th></tr></thead>
            <tbody>
```

```
            <tr th:each="i : ${cart.items}">
                <td><img class="thumb rounded me-2" th:src="${i.imageUrl}" alt=""><span
th:text="${i.name}"></span></td>
                <td class="text-end" th:text="${#numbers.formatDecimal(i.price, 0, 'POINT', 0,
'COMMA')} + ' ₫'"></td>
                <td>
                    <form th:action="@{/cart/update}" method="post" class="d-flex gap-1">
                        <input type="hidden" name="productId" th:value="${i.productId}">
                        <input type="number" name="quantity" min="0" class="form-control form-
control-sm" th:value="${i.quantity}">
                        <button class="btn btn-sm btn-outline-secondary">Cập nhật</button>
                    </form>
                </td>
                <td class="text-end" th:text="${#numbers.formatDecimal(i.subtotal, 0, 'POINT', 0,
'COMMA')} + ' ₫'"></td>
                <td>
                    <form th:action="@{/cart/remove/{id}(id=${i.productId})}" method="post">
                        <button class="btn btn-sm btn-outline-danger">✕</button>
                    </form>
                </td>
            </tr>
            </tbody>
            <tfoot>
            <tr><th colspan="3" class="text-end">Tổng cộng (<span
th:text="${cart.totalQuantity}"></span> sản phẩm)</th>
                <th class="text-end price" th:text="${#numbers.formatDecimal(cart.totalAmount, 0,
'POINT', 0, 'COMMA')} + ' ₫'"></th><th></th></tr>
            </tfoot>
        </table>

        <div class="d-flex justify-content-between">
            <form th:action="@{/cart/clear}" method="post"><button class="btn btn-outline-
danger">Xoá giỏ hàng</button></form>
            <div>
                <a class="btn btn-secondary" th:href="@{/products}">Tiếp tục mua</a>
                <a class="btn btn-success" th:href="@{/checkout}">Thanh toán →</a>
            </div>
        </div>
    </div>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

Bổ sung nút "Thêm vào giỏ" vào các trang của Phần 1:

`shop/index.html` – thay comment trong thẻ `<div class="mt-auto">`:

```
<form th:action="@{/cart/add}" method="post" class="d-inline">
    <input type="hidden" name="productId" th:value="${p.id}">
    <button class="btn btn-sm btn-primary" th:disabled="${p.quantity == 0}">🛒 Thêm vào giỏ</button>
</form>
<a class="btn btn-sm btn-outline-primary" th:href="@{/products/{id}(id=${p.id})}">Chi tiết</a>
```

`shop/detail.html` – thay comment *Form "Thêm vào giỏ"*:

```
<form th:action="@{/cart/add}" method="post" class="d-flex gap-2 mb-3" style="max-width:300px">
    <input type="hidden" name="productId" th:value="${product.id}">
    <input type="number" name="quantity" value="1" min="1" th:max="${product.quantity}"
```

```
class="form-control">
    <button class="btn btn-primary text-nowrap" th:disabled="${product.quantity == 0}">🛒 Thêm vào
giỏ</button>
</form>
```

`fragments/layout.html` – thêm vào navbar, ngay sau thẻ `</form>` tìm kiếm:

```
<ul class="navbar-nav ms-3">
    <li class="nav-item">
        <a class="nav-link" th:href="@{/cart}">Giỏ hàng
            <span class="badge bg-warning text-dark" th:text="${cartCount}">0</span></a>
    </li>
</ul>
```

và thêm 1 mục vào menu *Quản trị*: `<li><a class="dropdown-item" th:href="@{/admin/orders}">Đơn hàng</a></li>`

Bước 2.2 – Entity đơn hàng

`entity/PaymentMethod.java`

```
package com.example.eshop.entity;

public enum PaymentMethod { COD, BANKING }
```

`entity/OrderStatus.java`

```
package com.example.eshop.entity;

public enum OrderStatus { NEW, PAID, SHIPPING, DONE, CANCELLED }
```

`entity/Order.java`

```
package com.example.eshop.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import org.hibernate.annotations.CreationTimestamp;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "orders")          // "order" là từ khoá SQL nên bảng tên "orders"
@Getter @Setter @NoArgsConstructor
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @CreationTimestamp
    @Column(name = "order_date", updatable = false)
    private LocalDateTime orderDate;
```

```
    @Column(name = "customer_name", nullable = false, length = 100)
    private String customerName;

    @Column(nullable = false, length = 20)
    private String phone;

    @Column(nullable = false)
    private String address;

    private String note;

    @Enumerated(EnumType.STRING)
    @Column(name = "payment_method", nullable = false, length = 20)
    private PaymentMethod paymentMethod;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private OrderStatus status = OrderStatus.NEW;

    @Column(name = "total_amount", nullable = false, precision = 15, scale = 2)
    private BigDecimal totalAmount;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderDetail> details = new ArrayList<>();

    // (Phần 3 sẽ thêm thuộc tính user)

    public void addDetail(OrderDetail d) {
        d.setOrder(this);
        details.add(d);
    }
}
```

entity/OrderDetail.java

```
package com.example.eshop.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import java.math.BigDecimal;

@Entity
@Table(name = "order_detail")
@Getter @Setter @NoArgsConstructor
public class OrderDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id", nullable = false)
    private Order order;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "product_id", nullable = false)
```

```
    private Product product;

    @Column(nullable = false)
    private Integer quantity;

    @Column(name = "unit_price", nullable = false, precision = 15, scale = 2)
    private BigDecimal unitPrice;

    public BigDecimal getSubtotal() {
        return unitPrice.multiply(BigDecimal.valueOf(quantity));
    }
}
```

`cascade = ALL` trên `details`: chỉ cần `orderRepository.save(order)` là Hibernate tự INSERT các `order_detail`. `mappedBy = "order"`: phía `OrderDetail` giữ khoá ngoại.

`repository/OrderRepository.java`

```
package com.example.eshop.repository;

import com.example.eshop.entity.Order;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findAllByOrderByOrderDateDesc();
    // Phần 3: List<Order> findByUserUsernameOrderByOrderDateDesc(String username);
}
```

Bước 2.3 – Form thanh toán và OrderService

`dto/CheckoutForm.java`

```
package com.example.eshop.dto;

import com.example.eshop.entity.PaymentMethod;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.Getter;
import lombok.Setter;

@Getter @Setter
public class CheckoutForm {

    @NotBlank(message = "Vui lòng nhập họ tên")
    @Size(max = 100)
    private String customerName;

    @NotBlank(message = "Vui lòng nhập số điện thoại")
    @Pattern(regexp = "^0\\d{9}$", message = "Số điện thoại gồm 10 chữ số, bắt đầu bằng 0")
    private String phone;

    @NotBlank(message = "Vui lòng nhập địa chỉ giao hàng")
    @Size(max = 255)
```

```
    private String address;

    @Size(max = 500)
    private String note;

    @NotNull(message = "Chọn phương thức thanh toán")
    private PaymentMethod paymentMethod = PaymentMethod.COD;
}
```

service/OrderService.java

```
package com.example.eshop.service;

import com.example.eshop.cart.Cart;
import com.example.eshop.cart.CartItem;
import com.example.eshop.dto.CheckoutForm;
import com.example.eshop.entity.*;
import com.example.eshop.exception.NotFoundException;
import com.example.eshop.repository.OrderRepository;
import com.example.eshop.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.util.List;

@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orderRepository;
    private final ProductRepository productRepository;

    /**
     * Tạo đơn hàng từ giỏ. Toàn bộ trong 1 transaction:
     * nếu 1 sản phẩm không đủ hàng → ném exception → rollback, không trừ kho sản phẩm nào.
     */
    @Transactional
    public Order placeOrder(CheckoutForm form, Cart cart) {
        if (cart.isEmpty()) {
            throw new IllegalStateException("Giỏ hàng đang trống");
        }
        Order order = new Order();
        order.setCustomerName(form.getCustomerName());
        order.setPhone(form.getPhone());
        order.setAddress(form.getAddress());
        order.setNote(form.getNote());
        order.setPaymentMethod(form.getPaymentMethod());
        order.setStatus(OrderStatus.NEW);

        BigDecimal total = BigDecimal.ZERO;
        for (CartItem item : cart.getItems()) {
            Product p = productRepository.findById(item.getProductId())
                    .orElseThrow(() -> new IllegalStateException("Sản phẩm \"" + item.getName() +
"\" không còn bán"));
            if (p.getQuantity() < item.getQuantity()) {
                throw new IllegalStateException("\"" + p.getName() + "\" chỉ còn " +
p.getQuantity() + " sản phẩm");
```

```
            }
            p.setQuantity(p.getQuantity() - item.getQuantity());   // dirty checking → UPDATE
product

            OrderDetail d = new OrderDetail();
            d.setProduct(p);
            d.setQuantity(item.getQuantity());
            d.setUnitPrice(p.getPrice());                          // giá lấy từ DB, không tin giá
trong giỏ
            order.addDetail(d);
            total = total.add(d.getSubtotal());
        }
        order.setTotalAmount(total);
        return orderRepository.save(order);
    }

    public Order findById(Long id) {
        return orderRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("Không tìm thấy đơn hàng #" + id));
    }

    public List<Order> findAll() {
        return orderRepository.findAllByOrderByOrderDateDesc();
    }

    /** Giả lập cổng thanh toán xác nhận đã nhận tiền. */
    @Transactional
    public void markPaid(Long id) {
        Order o = findById(id);
        if (o.getStatus() == OrderStatus.NEW) {
            o.setStatus(OrderStatus.PAID);
        }
    }

    @Transactional
    public void updateStatus(Long id, OrderStatus status) {
        Order o = findById(id);
        // Huỷ đơn → hoàn lại tồn kho
        if (status == OrderStatus.CANCELLED && o.getStatus() != OrderStatus.CANCELLED) {
            o.getDetails().forEach(d -> d.getProduct().setQuantity(d.getProduct().getQuantity() +
d.getQuantity()));
        }
        o.setStatus(status);
    }

    @Transactional
    public void delete(Long id) {
        orderRepository.delete(findById(id));   // cascade xoá order_detail
    }
}
```

Bước 2.4 – Controller thanh toán và đơn hàng

`controller/CheckoutController.java`

```
package com.example.eshop.controller;

import com.example.eshop.cart.Cart;
```

```
import com.example.eshop.dto.CheckoutForm;
import com.example.eshop.entity.Order;
import com.example.eshop.entity.PaymentMethod;
import com.example.eshop.service.OrderService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/checkout")
@RequiredArgsConstructor
public class CheckoutController {

    private final Cart cart;
    private final OrderService orderService;

    @GetMapping
    public String form(Model model) {
        if (cart.isEmpty()) return "redirect:/cart";
        model.addAttribute("form", new CheckoutForm());
        model.addAttribute("cart", cart);
        return "checkout/form";
    }

    @PostMapping
    public String submit(@Valid @ModelAttribute("form") CheckoutForm form, BindingResult result,
                         Model model, RedirectAttributes ra) {
        if (result.hasErrors()) {
            model.addAttribute("cart", cart);
            return "checkout/form";
        }
        try {
            Order order = orderService.placeOrder(form, cart);
            cart.clear();
            if (order.getPaymentMethod() == PaymentMethod.BANKING) {
                return "redirect:/orders/" + order.getId() + "/payment";
            }
            ra.addFlashAttribute("message", "Đặt hàng thành công! Mã đơn #" + order.getId());
            return "redirect:/orders/" + order.getId();
        } catch (IllegalStateException e) {
            ra.addFlashAttribute("error", e.getMessage());
            return "redirect:/cart";
        }
    }
}
```

`controller/OrderController.java` (phiên bản Phần 2)

```
package com.example.eshop.controller;

import com.example.eshop.service.OrderService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
```

```
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/orders")
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;

    @GetMapping("/{id}")
    public String detail(@PathVariable Long id, Model model) {
        model.addAttribute("order", orderService.findById(id));
        return "order/detail";
    }

    @GetMapping("/{id}/payment")
    public String payment(@PathVariable Long id, Model model) {
        model.addAttribute("order", orderService.findById(id));
        return "order/payment";
    }

    @PostMapping("/{id}/pay")
    public String pay(@PathVariable Long id, RedirectAttributes ra) {
        orderService.markPaid(id);
        ra.addFlashAttribute("message", "Thanh toán thành công đơn #" + id);
        return "redirect:/orders/" + id;
    }
}
```

`controller/admin/AdminOrderController.java` – người bán quản lý đơn hàng

```
package com.example.eshop.controller.admin;

import com.example.eshop.entity.OrderStatus;
import com.example.eshop.service.OrderService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/admin/orders")
@RequiredArgsConstructor
public class AdminOrderController {

    private final OrderService orderService;

    @GetMapping
    public String list(Model model) {
        model.addAttribute("orders", orderService.findAll());
        return "admin/order/list";
    }

    @GetMapping("/{id}")
    public String detail(@PathVariable Long id, Model model) {
        model.addAttribute("order", orderService.findById(id));
        model.addAttribute("statuses", OrderStatus.values());
        return "admin/order/detail";
```

```
    }

    @PostMapping("/{id}/status")
    public String updateStatus(@PathVariable Long id, @RequestParam OrderStatus status,
RedirectAttributes ra) {
        orderService.updateStatus(id, status);
        ra.addFlashAttribute("message", "Đã cập nhật trạng thái đơn #" + id);
        return "redirect:/admin/orders/" + id;
    }

    @PostMapping("/{id}/delete")
    public String delete(@PathVariable Long id, RedirectAttributes ra) {
        orderService.delete(id);
        ra.addFlashAttribute("message", "Đã xoá đơn #" + id);
        return "redirect:/admin/orders";
    }
}
```

Bước 2.5 – Template thanh toán và đơn hàng

`templates/checkout/form.html`

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Thanh toán')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <h3>Thanh toán</h3>
    <div class="row g-4">
        <div class="col-md-7">
            <form th:action="@{/checkout}" th:object="${form}" method="post">
                <div class="mb-3">
                    <label class="form-label">Họ tên người nhận *</label>
                    <input class="form-control" th:field="*{customerName}" th:errorclass="is-
invalid">
                    <div class="invalid-feedback" th:errors="*{customerName}"></div>
                </div>
                <div class="mb-3">
                    <label class="form-label">Số điện thoại *</label>
                    <input class="form-control" th:field="*{phone}" th:errorclass="is-invalid">
                    <div class="invalid-feedback" th:errors="*{phone}"></div>
                </div>
                <div class="mb-3">
                    <label class="form-label">Địa chỉ giao hàng *</label>
                    <input class="form-control" th:field="*{address}" th:errorclass="is-invalid">
                    <div class="invalid-feedback" th:errors="*{address}"></div>
                </div>
                <div class="mb-3">
                    <label class="form-label">Ghi chú</label>
                    <textarea class="form-control" rows="2" th:field="*{note}"></textarea>
                </div>
                <div class="mb-3">
                    <label class="form-label d-block">Phương thức thanh toán *</label>
                    <div class="form-check">
                        <input class="form-check-input" type="radio" th:field="*{paymentMethod}"
value="COD" id="pmCod">
                        <label class="form-check-label" for="pmCod">Thanh toán khi nhận hàng
```

```
(COD)</label>
                    </div>
                    <div class="form-check">
                        <input class="form-check-input" type="radio" th:field="*{paymentMethod}"
value="BANKING" id="pmBank">
                        <label class="form-check-label" for="pmBank">Chuyển khoản ngân hàng (giả
lập)</label>
                    </div>
                </div>
                <button class="btn btn-success btn-lg">Đặt hàng</button>
                <a class="btn btn-link" th:href="@{/cart}">← Quay lại giỏ</a>
            </form>
        </div>

        <div class="col-md-5">
            <div class="card">
                <div class="card-header">Đơn hàng của bạn</div>
                <ul class="list-group list-group-flush">
                    <li class="list-group-item d-flex justify-content-between" th:each="i :
${cart.items}">
                        <span><span th:text="${i.name}"></span> × <b
th:text="${i.quantity}"></b></span>
                        <span th:text="${#numbers.formatDecimal(i.subtotal, 0, 'POINT', 0,
'COMMA')} + ' ₫'"></span>
                    </li>
                    <li class="list-group-item d-flex justify-content-between fw-bold">
                        <span>Tổng cộng</span>
                        <span class="price" th:text="${#numbers.formatDecimal(cart.totalAmount, 0,
'POINT', 0, 'COMMA')} + ' ₫'"></span>
                    </li>
                </ul>
            </div>
        </div>
    </div>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

`templates/order/payment.html` – trang chuyển khoản giả lập

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Chuyển khoản')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container text-center" style="max-width:520px">
    <h3>Thanh toán đơn #<span th:text="${order.id}"></span></h3>
    <img src="https://placehold.co/240x240?text=QR+CODE" class="my-3 border rounded" alt="QR">
    <table class="table text-start">
        <tr><th>Ngân hàng</th><td>Vietcombank – CN TP.HCM</td></tr>
        <tr><th>Số tài khoản</th><td>0123 456 789</td></tr>
        <tr><th>Chủ tài khoản</th><td>CONG TY ESHOP</td></tr>
        <tr><th>Số tiền</th><td class="price" th:text="${#numbers.formatDecimal(order.totalAmount,
0, 'POINT', 0, 'COMMA')} + ' ₫'"></td></tr>
        <tr><th>Nội dung CK</th><td th:text="'ESHOP' + ${order.id}"></td></tr>
    </table>
    <form th:action="@{/orders/{id}/pay(id=${order.id})}" method="post">
        <button class="btn btn-success btn-lg">Tôi đã chuyển khoản</button>
```

```
    </form>
    <p class="text-muted small mt-2">(Demo: bấm nút để giả lập ngân hàng xác nhận đã nhận tiền)</p>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

templates/order/detail.html – chi tiết đơn (khách hàng xem)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Đơn hàng')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>
    <h3>Đơn hàng #<span th:text="${order.id}"></span>
        <span class="badge bg-info text-dark" th:text="${order.status}"></span></h3>
    <p>
        Ngày đặt: <span th:text="${#temporals.format(order.orderDate, 'dd/MM/yyyy
HH:mm')}"></span><br>
        Người nhận: <b th:text="${order.customerName}"></b> — <span
th:text="${order.phone}"></span><br>
        Địa chỉ: <span th:text="${order.address}"></span><br>
        Thanh toán: <span th:text="${order.paymentMethod}"></span>
    </p>
    <table class="table">
        <thead><tr><th>Sản phẩm</th><th class="text-end">Đơn giá</th><th class="text-
end">SL</th><th class="text-end">Thành tiền</th></tr></thead>
        <tbody>
        <tr th:each="d : ${order.details}">
            <td th:text="${d.product.name}"></td>
            <td class="text-end" th:text="${#numbers.formatDecimal(d.unitPrice, 0, 'POINT', 0,
'COMMA')}"></td>
            <td class="text-end" th:text="${d.quantity}"></td>
            <td class="text-end" th:text="${#numbers.formatDecimal(d.subtotal, 0, 'POINT', 0,
'COMMA')}"></td>
        </tr>
        </tbody>
        <tfoot><tr><th colspan="3" class="text-end">Tổng</th>
            <th class="text-end price" th:text="${#numbers.formatDecimal(order.totalAmount, 0,
'POINT', 0, 'COMMA')} + ' ₫'"></th></tr></tfoot>
    </table>
    <a th:if="${order.paymentMethod.name() == 'BANKING' and order.status.name() == 'NEW'}"
       class="btn btn-warning" th:href="@{/orders/{id}/payment(id=${order.id})}">Thanh toán
ngay</a>
    <a class="btn btn-primary" th:href="@{/products}">Tiếp tục mua sắm</a>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

templates/admin/order/list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Quản lý đơn hàng')}"></head>
```

```
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>
    <h3>Đơn hàng</h3>
    <table class="table table-bordered table-hover align-middle">
        <thead class="table-light"><tr><th>#</th><th>Ngày</th><th>Khách</th><th>SĐT</th><th>Thanh
toán</th><th>Trạng thái</th><th class="text-end">Tổng</th><th></th></tr></thead>
        <tbody>
        <tr th:each="o : ${orders}">
            <td th:text="${o.id}"></td>
            <td th:text="${#temporals.format(o.orderDate, 'dd/MM/yyyy HH:mm')}"></td>
            <td th:text="${o.customerName}"></td>
            <td th:text="${o.phone}"></td>
            <td th:text="${o.paymentMethod}"></td>
            <td><span class="badge bg-secondary" th:text="${o.status}"></span></td>
            <td class="text-end" th:text="${#numbers.formatDecimal(o.totalAmount, 0, 'POINT', 0,
'COMMA')}"></td>
            <td><a class="btn btn-sm btn-outline-primary"
th:href="@{/admin/orders/{id}(id=${o.id})}">Xem</a></td>
        </tr>
        </tbody>
    </table>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

templates/admin/order/detail.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Chi tiết đơn')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>
    <h3>Đơn #<span th:text="${order.id}"></span></h3>
    <p>Khách: <b th:text="${order.customerName}"></b> — <span th:text="${order.phone}"></span> —
<span th:text="${order.address}"></span><br>
       Ghi chú: <span th:text="${order.note}"></span></p>

    <table class="table">
        <thead><tr><th>Sản phẩm</th><th class="text-end">Đơn giá</th><th class="text-
end">SL</th><th class="text-end">Thành tiền</th></tr></thead>
        <tbody><tr th:each="d : ${order.details}">
            <td th:text="${d.product.name}"></td>
            <td class="text-end" th:text="${#numbers.formatDecimal(d.unitPrice, 0, 'POINT', 0,
'COMMA')}"></td>
            <td class="text-end" th:text="${d.quantity}"></td>
            <td class="text-end" th:text="${#numbers.formatDecimal(d.subtotal, 0, 'POINT', 0,
'COMMA')}"></td>
        </tr></tbody>
        <tfoot><tr><th colspan="3" class="text-end">Tổng</th>
            <th class="text-end" th:text="${#numbers.formatDecimal(order.totalAmount, 0, 'POINT',
0, 'COMMA')} + ' ₫'"></th></tr></tfoot>
    </table>

    <form th:action="@{/admin/orders/{id}/status(id=${order.id})}" method="post" class="d-flex gap-
```

```
2 mb-3">
        <select name="status" class="form-select w-auto">
            <option th:each="s : ${statuses}" th:value="${s}" th:text="${s}" th:selected="${s ==
order.status}"></option>
        </select>
        <button class="btn btn-primary">Cập nhật trạng thái</button>
    </form>
    <form th:action="@{/admin/orders/{id}/delete(id=${order.id})}" method="post" onsubmit="return
confirm('Xoá đơn này?')">
        <button class="btn btn-outline-danger">Xoá đơn</button>
        <a class="btn btn-secondary" th:href="@{/admin/orders}">← Danh sách</a>
    </form>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

Bước 2.6 – Kiểm thử Phần 2

#	Thao tác	Kết quả mong đợi
1	Thêm 2 sản phẩm vào giỏ	Badge trên navbar hiện tổng số lượng
2	Mở thêm 1 trình duyệt khác (hoặc cửa sổ ẩn danh)	Giỏ hàng trống – mỗi session một giỏ
3	Sửa số lượng > tồn kho	Báo lỗi *chỉ còn n*
4	Sửa số lượng = 0	Dòng đó bị xoá khỏi giỏ
5	Thanh toán, để trống SĐT / nhập "123"	Báo lỗi validation
6	Thanh toán COD	Trang đơn hàng; bảng `orders`, `order_detail` có dữ liệu; `product.quantity` giảm
7	Thanh toán BANKING → *Tôi đã chuyển khoản*	Trạng thái đơn `PAID`
8	`/admin/orders` → đổi trạng thái `CANCELLED`	Tồn kho được cộng trả lại
9	Trong DB đặt `quantity = 0` cho 1 sản phẩm đang có trong giỏ rồi đặt hàng	Báo lỗi, **không** sản phẩm nào bị trừ kho (rollback)

PHẦN 3 – Spring Security + JWT + 3 vai trò

3.1. Lý thuyết nền tảng (đọc kỹ trước khi code)

Mục này giải thích **từng khái niệm bằng ví dụ đời thường và ví dụ chạy thật trên eShop**. Mỗi khái niệm đều được dùng lại ở các bước 3.2 → 3.13, nên nắm được mục này thì đọc code sẽ thấy rất "thẳng".

3.1.1. Vì sao cần bảo mật? – Thử "tấn công" eShop của Phần 2

Ở cuối Phần 2, ứng dụng **chưa có bảo mật**. Thử 3 việc sau (ai cũng làm được, không cần tài khoản):

#	Thao tác	Kết quả ở Phần 2	Vấn đề
1	Gõ thẳng `http://localhost:8080/admin/products`	Vào được trang quản trị	Ai cũng sửa giá, xoá hàng được
2	Mở F12 → Console, chạy `fetch('/admin/products/1/delete', {method:'POST'})`	Sản phẩm #1 bị xoá	Chỉ cần biết URL là phá được
3	Mở `http://localhost:8080/orders/1`	Xem được tên, SĐT, địa chỉ của khách	Lộ dữ liệu cá nhân

Ẩn nút "Quản trị" trên menu không giải quyết được gì, vì kẻ xấu không cần bấm nút, họ gõ thẳng URL hoặc gửi request bằng Postman/curl. Bảo mật thật phải nằm ***ở server***, kiểm tra **mọi request** trước khi nó tới Controller. Đó là việc của Spring Security.

3.1.2. Hai câu hỏi cốt lõi: Authentication và Authorization

Ví dụ đời thường – khách sạn:

```
Bạn đến quầy lễ tân, đưa CCCD ──► Lễ tân đối chiếu ảnh, tên với danh sách đặt phòng
                                        → "Đúng là anh Nguyễn Văn Mua"
                        ① AUTHENTICATION (xác thực)
 Lễ tân đưa thẻ từ phòng 302  ──► Thẻ mở được phòng 302, hồ bơi; KHÔNG mở được phòng 303, kho,
phòng giám đốc
                        ② AUTHORIZATION (phân quyền)
```

	Authentication – *Bạn là ai?*	**Authorization – *Bạn được làm gì?***
Khách sạn	Kiểm tra CCCD	Thẻ từ mở được những cửa nào
eShop	Kiểm tra `username` + `password` với bảng `users`	`customer1` vào được `/cart`, không vào được `/admin/**`
Thất bại → mã lỗi	**401 Unauthorized** (*"tôi chưa biết bạn là ai"*). Web: chuyển về `/login`	**403 Forbidden** (*"tôi biết bạn là ai, nhưng bạn không được phép"*). Web: trang `/403`
Diễn ra khi	1 lần lúc đăng nhập (sau đó nhờ JWT để "nhớ")	Ở **mọi** request

Ví dụ trên eShop (sẽ kiểm tra lại ở cuối bài):

Người gửi request	Request	Kết quả	Vì sao
Chưa đăng nhập	`GET`	302 → `/login`	Chưa xác thực (lỗi loại 401)

Người gửi request	Request	Kết quả	Vì sao
	`/admin/products`		
`customer1`	`GET /admin/products`	**403**	Đã xác thực nhưng thiếu quyền SELLER
`seller1`	`GET /admin/products`	200	Đủ quyền
Gõ sai mật khẩu	`POST /login`	quay lại `/login?error=bad`	Xác thực thất bại

3.1.3. Principal, Authority, Role – Spring "nhìn" người dùng như thế nào?

Sau khi đăng nhập thành công, Spring Security tạo một đối tượng **`Authentication`** và cất vào **`SecurityContextHolder`** (một "ngăn kéo" gắn với luồng xử lý request hiện tại). Với `customer1`, đối tượng đó trông như sau:

```
Authentication (UsernamePasswordAuthenticationToken)
├── principal     : UserDetails { username = "customer1", password = "$2a$10$...", enabled = true }
├── credentials   : null                    ← mật khẩu bị xoá khỏi bộ nhớ sau khi xác thực xong
├── authorities   : [ ROLE_CUSTOMER ]       ← danh sách quyền (GrantedAuthority)
└── authenticated : true
```

- **Principal** = "người đang dùng hệ thống", trong bài là đối tượng `UserDetails` do ta tạo từ bảng `users`.
- **GrantedAuthority** = một **chuỗi quyền** bất kỳ, ví dụ `ROLE_CUSTOMER`, `product:delete`.
- **Role** = authority **bắt đầu bằng `ROLE_`**. Spring quy ước: `hasRole("SELLER")` ≡ `hasAuthority("ROLE_SELLER")`.

Trong DB (`users.role`)	Trong code Java/Thymeleaf	Đúng/Sai
`ROLE_SELLER`	`hasRole("SELLER")`	✅
`ROLE_SELLER`	`hasAuthority("ROLE_SELLER")`	✅
`ROLE_SELLER`	`hasRole("ROLE_SELLER")`	❌ Spring tìm `ROLE_ROLE_SELLER`
`SELLER`	`hasRole("SELLER")`	❌ thiếu tiền tố `ROLE_`

Ở bất kỳ đâu trong code, ta lấy được người dùng hiện tại:

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
auth.getName();          // "customer1"
auth.getAuthorities();   // [ROLE_CUSTOMER]
// Trong Controller viết gọn hơn: tham số Principal principal → principal.getName()
```

3.1.4. Ba vai trò của eShop – và "khách" là ai?

Vai trò	Ai	Có dòng trong bảng `users`?	Authority
SELLER	Người bán / quản trị	Có (`role = 'ROLE_SELLER'`)	`ROLE_SELLER`
CUSTOMER	Khách hàng đã có tài khoản	Có (`role = 'ROLE_CUSTOMER'`)	`ROLE_CUSTOMER`
GUEST	Người vào web nhưng **chưa đăng nhập**	**Không**	`ROLE_GUEST` (do ta cấu hình)

Người chưa đăng nhập vẫn được Spring Security gắn một `Authentication` đặc biệt tên là **`AnonymousAuthenticationToken`** (mặc định mang quyền `ROLE_ANONYMOUS`). Ta đổi quyền mặc định đó thành `ROLE_GUEST`:

```
http.anonymous(a -> a.authorities("ROLE_GUEST"));
```

Nhờ vậy cả 3 vai trò đều viết thống nhất bằng `hasRole(...)`:

```
.requestMatchers("/register").hasRole("GUEST")          // chỉ người CHƯA đăng nhập mới đăng ký
.requestMatchers("/cart/**").hasRole("CUSTOMER")
.requestMatchers("/admin/**").hasRole("SELLER")
```

Khi người đăng ký xong và đăng nhập, họ **không còn là GUEST** mà trở thành CUSTOMER. Vì vậy CUSTOMER vào `/register` sẽ nhận 403, đúng với yêu cầu "khách: đăng ký tài khoản mới".

3.1.5. Mật khẩu: vì sao phải dùng BCrypt?

Không bao giờ lưu mật khẩu thô. Nếu file backup DB bị lộ, kẻ xấu có ngay mật khẩu của mọi người, và nhiều người dùng chung mật khẩu cho Gmail, ngân hàng…

Thử so sánh 3 cách lưu mật khẩu `123456`:

Cách lưu	Giá trị lưu trong cột `password`	Nhận xét
Thô	`123456`	❌ Lộ ngay
MD5	`e10adc3949ba59abbe56e057f20f883e`	❌ Lúc nào cũng ra đúng chuỗi này → tra Google là ra "123456" (bảng tra sẵn / *rainbow table*); máy tính băm được hàng tỷ MD5 mỗi giây
BCrypt	`$2a$10$uPUNNrC1JUnzBYDZD6AQBe.swyBhdnARmWl.Ctpo737mZZF4hejyS`	✅ Có *salt* ngẫu nhiên + cố tình chậm

Cùng mật khẩu `123456`, mỗi lần băm BCrypt cho một kết quả khác nhau (đây là 3 kết quả thật):

```
$2a$10$uPUNNrC1JUnzBYDZD6AQBe.swyBhdnARmWl.Ctpo737mZZF4hejyS   ← đang dùng trong eShop.sql
$2a$10$DzDtE6sP4T7n/zry4hM49eVMlYDo0STpQ7FicWUTw5UTBegd8D9Iy
$2a$10$sjA6dzYg9NixIi0EpLxFpOGJS7NWRr21FAUIIJRu/GmIOtgWEP.nu
```

Giải phẫu một chuỗi BCrypt (60 ký tự):

```
$2a$ 10 $ uPUNNrC1JUnzBYDZD6AQBe . swyBhdnARmWl.Ctpo737mZZF4hejyS
  |    |          |                        |
  |    |          |                        └─ 31 ký tự: kết quả băm
  |    |          └─ 22 ký tự: SALT ngẫu nhiên (lưu ngay trong chuỗi)
  |    └─ cost = 10 → lặp 2^10 = 1024 vòng (càng lớn càng chậm, càng khó dò)
  └─ phiên bản thuật toán
```

Vì mỗi lần băm cho kết quả khác nhau, **không được so sánh chuỗi** mà phải dùng `matches()`. Hàm này đọc salt trong chuỗi đã lưu, băm lại mật khẩu người dùng nhập với đúng salt đó rồi so sánh:

```
PasswordEncoder encoder = new BCryptPasswordEncoder();
String hash = encoder.encode("123456");          // lúc ĐĂNG KÝ: lưu hash vào DB
encoder.matches("123456", hash);                 // lúc ĐĂNG NHẬP: true
encoder.matches("1234567", hash);                // false
hash.equals(encoder.encode("123456"));           // false !!! – đừng làm thế này
```

3.1.6. Spring Security là một chuỗi Filter (bộ lọc)

Ví dụ đời thường – cửa an ninh sân bay: hành khách phải đi qua lần lượt nhiều trạm: kiểm tra vé → kiểm tra giấy tờ → soi hành lý → cổng lên máy bay. Trạm nào không đạt thì bị chặn lại, không đi tiếp được.

Spring Security làm đúng như vậy: nó gắn một **chuỗi Filter** đứng **trước** `DispatcherServlet`. Mọi request phải đi qua chuỗi này rồi mới tới Controller.

```
Trình duyệt / Postman
      |  HTTP request
      ▼
┌──────────────────────────── Spring Security (SecurityFilterChain) ────────────────────────────┐
| ① SecurityContextHolderFilter   chuẩn bị "ngăn kéo" SecurityContext (lúc đầu rỗng)          |
| ② CsrfFilter                    POST/PUT/DELETE phải kèm _csrf token đúng (web)             |
| ③ LogoutFilter                  nếu là POST /logout → xoá cookie, huỷ session, xong         |
| ④ JwtAuthenticationFilter  ★    TA TỰ VIẾT: đọc JWT → hợp lệ → đặt Authentication vào ngăn kéo
|
| ⑤ AnonymousAuthenticationFilter ngăn kéo vẫn rỗng? → đặt Anonymous (ROLE_GUEST)             |
| ⑥ ExceptionTranslationFilter    "bắt lỗi" của ⑦: chưa đăng nhập → /login (401); thiếu quyền →
/403 |
| ⑦ AuthorizationFilter           so khớp luật authorizeHttpRequests: cho qua hay chặn?       |
└──────────────────────────────────────────────────────────────────────────────────────────────┘
      |  được phép
      ▼
DispatcherServlet ──► Controller ──► Service (@PreAuthorize kiểm tra thêm lần nữa) ──► Repository
```

Muốn xem chuỗi filter thật, đặt `logging.level.org.springframework.security=DEBUG` (bước 3.2). Khi ứng dụng khởi động, console in danh sách filter; mỗi request in ra filter nào đang xử lý và kết quả ra sao.

3.1.7. Ứng dụng "nhớ" người đã đăng nhập bằng cách nào? Session và Token

HTTP là giao thức **không trạng thái (stateless)**: server không tự nhớ request trước là của ai. Sau khi đăng nhập, mỗi request tiếp theo phải mang theo "bằng chứng" là mình đã đăng nhập. Có 2 cách:

Ví dụ đời thường – gửi xe:

	Cách 1: Session – vé xe có số	**Cách 2: JWT – thẻ ra vào có chữ ký**
Hình dung	Bãi xe đưa bạn vé số **A123**, đồng thời ghi vào sổ: *A123 = xe 59X1-12345*. Lấy xe phải đưa vé, bảo vệ **tra sổ**	Công ty cấp thẻ in sẵn *"Nguyễn Văn Mua – Phòng Kinh doanh – hạn 31/12"* **có chữ ký và con dấu giám đốc**. Bảo vệ chỉ cần **kiểm tra chữ ký**, không cần tra sổ
Server lưu gì	Lưu thông tin người dùng trong `HttpSession` (bộ nhớ server)	**Không lưu gì**
Client giữ gì	Cookie `JSESSIONID=8F3A...` (chỉ là một con số)	Chuỗi JWT chứa sẵn username, quyền, hạn dùng
Có nhiều server	Server B không có "sổ" của server A → phải chia sẻ session	Server nào có khoá bí mật cũng kiểm tra được
Đăng xuất / thu hồi	Xoá dòng trong sổ là xong	Khó: token vẫn hợp lệ tới khi hết hạn → đặt hạn ngắn
Hợp với	Web truyền thống 1 server	REST API, mobile app, nhiều server/microservice

Bài này dùng **JWT** cho cả web và API. Riêng giỏ hàng vẫn để trong `HttpSession` như Phần 2, vì đó là dữ liệu tạm của ứng dụng, không phải thông tin đăng nhập.

3.1.8. JWT (JSON Web Token)

Đây là token **thật** mà eShop cấp cho `customer1` (dùng đúng khoá `app.jwt.secret` trong bước 3.2). Token gồm 3 phần, cách nhau bằng dấu chấm:

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJjdXN0b21lcjEiLCJyb2xlcyI6IlJPTEVfQ1VTVE9NRVIiLCJpYXQiOjE3OTI0NTgwMD
AsImV4cCI6MTc5MjU0NDQwMH0.EqumqvKXzkgfo-s-Mpiwyneknic2l0TXeU3HDO94d2A
└──── HEADER ─────┘ └──────────────────────────────────── PAYLOAD
──────────────────────────────────────────────────────┘ └────────── SIGNATURE ──────────────────┘
```

Giải mã Base64URL hai phần đầu (dán token vào https://jwt.io để tự kiểm chứng):

```
// HEADER – thuật toán ký
{ "alg": "HS256" }

// PAYLOAD – các "claim" (thông tin)
{
  "sub":   "customer1",        // subject: token này của ai
  "roles": "ROLE_CUSTOMER",    // claim tự đặt: vai trò
```

```
  "iat":   1792458000,         // issued at: phát hành lúc 20/10/2026 08:00 (giờ
VN)
  "exp":   1792544400          // expiration: hết hạn sau đúng 24 giờ (86 400
giây)
}
```

```
// SIGNATURE – chữ ký
HMAC-SHA256( "eyJhbGciOiJIUzI1NiJ9" + "." + "eyJzdWIiOiJjdXN0b21l...", app.jwt.secret )
   = EqumqvKXzkgfo-s-Mpiwyneknic2l0TXeU3HDO94d2A
```

Hai điều quan trọng nhất về JWT:

1. **Payload chỉ được mã hoá Base64, KHÔNG được mã hoá bí mật.** Ai cầm token cũng đọc được nội dung. Vì vậy **tuyệt đối không** đặt mật khẩu, số thẻ… vào payload.
2. **Chữ ký chống sửa đổi.** Thử làm kẻ gian: `customer1` sửa payload `"roles":"ROLE_CUSTOMER"` thành `"roles":"ROLE_SELLER"` để thành người bán:

```
Payload sau khi sửa (Base64):
eyJzdWIiOiJjdXN0b21lcjEiLCJyb2xlcyI6IlJPTEVfU0VMTEVSIiwiaWF0IjoxNzkyNDU4MDAwLCJleHAiOjE3OTI1NDQ0MDB
9
Chữ ký cũ gửi kèm:            EqumqvKXzkgfo-s-Mpiwyneknic2l0TXeU3HDO94d2A
Server tính lại chữ ký:       0YedPuD-_HKbQOWJcqQ9fZWHZam0ltUaCuCJiggso_w     ≠  chữ ký gửi kèm
→ jjwt ném SignatureException → JwtAuthenticationFilter bỏ qua token → người gửi bị coi là GUEST
```

Muốn tạo được chữ ký đúng thì phải biết `app.jwt.secret`, mà khoá này **chỉ server biết**. Đó là lý do khoá phải dài (≥ 256 bit) và không được đưa lên GitHub công khai.

Các trường hợp token bị từ chối và lỗi tương ứng trong thư viện jjwt:

Tình huống	Exception của jjwt	eShop xử lý
Sửa payload / chữ ký sai	`SignatureException`	Coi như GUEST
Quá thời điểm `exp`	`ExpiredJwtException`	Coi như GUEST → phải đăng nhập lại
Chuỗi không đúng định dạng 3 phần	`MalformedJwtException`	Coi như GUEST
Token hợp lệ nhưng tài khoản vừa bị khoá	–	Filter nạp lại user từ DB, thấy `enabled = false` → GUEST

3.1.9. Token được gửi lên server bằng cách nào?

eShop có 2 loại client nên dùng 2 cách gửi token. `JwtAuthenticationFilter` (bước 3.6) đọc được cả hai:

(a) REST API (Postman, mobile app): header `Authorization`

```
DELETE /api/products/5 HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJzZWxsZXIx...
```

Client tự lưu token sau khi gọi `POST /api/auth/login`, rồi tự gắn header vào mỗi request.

(b) Web Thymeleaf: cookie HttpOnly

Trang web dùng thẻ `<a href>` và `<form>` thông thường, nên không tự gắn header được. Vì vậy lúc đăng nhập, server trả JWT về trong một **cookie**, và trình duyệt **tự động gửi cookie này kèm mọi request**:

```
HTTP/1.1 302 Found      ← phản hồi của POST /login
Location: /
Set-Cookie: ACCESS_TOKEN=eyJhbGciOiJIUzI1NiJ9...; Path=/; Max-Age=86400;
HttpOnly; SameSite=Lax

GET /cart HTTP/1.1      ← các request sau, trình duyệt tự gửi kèm
Cookie: ACCESS_TOKEN=eyJhbGciOiJIUzI1NiJ9...; JSESSIONID=8F3A...
```

Thuộc tính cookie	Ý nghĩa	Chống lại
`HttpOnly`	JavaScript (`document.cookie`) **không đọc được**	Mã độc XSS đánh cắp token
`SameSite=Lax`	Trang web khác gửi POST tới eShop thì trình duyệt **không** kèm cookie	Một phần CSRF
`Max-Age=86400`	Cookie tự xoá sau 24 giờ, bằng hạn của token	
`Secure` (khi có HTTPS)	Chỉ gửi qua kết nối mã hoá	Nghe lén trên mạng Wi-Fi

3.1.10. CSRF – vì sao web vẫn phải bật, còn API thì tắt?

Kịch bản tấn công: `seller1` đang đăng nhập eShop (trình duyệt đang giữ cookie `ACCESS_TOKEN`). Kẻ xấu gửi cho anh ta link tới một trang "khuyến mãi" có mã ẩn như sau:

```
<!-- trang độc hại: http://khuyen-mai-hot.xyz -->
<form action="http://localhost:8080/admin/products/1/delete" method="post" id="f"></form>
<script>document.getElementById('f').submit();</script>
```

Vì cookie được trình duyệt **tự động** gửi kèm, nếu không có biện pháp phòng thủ thì eShop sẽ tưởng chính `seller1` bấm Xoá. Kiểu tấn công này gọi là **CSRF** (*Cross-Site Request Forgery*, giả mạo request từ trang khác).

Cách Spring chống: mỗi form của eShop có thêm một ô ẩn chứa **CSRF token**. Đây là một chuỗi ngẫu nhiên do server sinh ra, trang của kẻ xấu không thể biết:

```
<!-- Thymeleaf th:action tự sinh ra dòng input ẩn này -->
<form action="/admin/products/1/delete" method="post">
    <input type="hidden" name="_csrf" value="kX9f2Lq...rất-dài-và-ngẫu-nhiên">
    <button>Xoá</button>
</form>
```

`CsrfFilter` từ chối mọi POST/PUT/DELETE thiếu token hoặc token sai, trả về **403**. Form của kẻ xấu không có token đúng nên bị chặn.

Cookie `SameSite=Lax` (mục 3.1.9) cũng đã chặn phần lớn kiểu tấn công này trên trình duyệt hiện đại. CSRF token là **lớp bảo vệ thứ hai**, vẫn có tác dụng với trình duyệt cũ hoặc khi cookie bị cấu hình sai.

	Web (`webFilterChain`)	**API (`apiFilterChain`)**
Token đăng nhập nằm ở	Cookie → trình duyệt **tự gửi**	Header `Authorization` → client phải **tự gắn**
Trang lạ có lợi dụng được không?	Có thể	Không (trang lạ không biết token để gắn header)
CSRF	**Bật** (mặc định)	**Tắt**: `csrf.disable()`

Hệ quả khi code: mọi form POST phải viết `th:action="@{...}"` (không phải `action="..."`), và **đăng xuất phải là form POST**, không dùng link `<a href="/logout">`.

3.1.11. Ba tầng phân quyền

Tầng	**Viết ở đâu**	**Ví dụ trong eShop**	**Ý nghĩa**
① **URL**	`SecurityConfig`	`.requestMatchers("/admin/**").hasRole("SELLER")`	"Cổng bảo vệ" – chặn ngay ở filter, chưa tới Controller
② **Method**	Service	`@PreAuthorize("hasRole('SELLER')") public void delete(...)`	"Khoá két" – dù vào bằng đường nào (web, API, quên khai báo URL) thì cũng không gọi được
③ **View**	Thymeleaf	`<li sec:authorize="hasRole('SELLER')">Quản trị</li>`	"Biển chỉ dẫn" – chỉ ẩn/hiện cho đẹp, **không** phải bảo mật

Luật URL được xét từ trên xuống, gặp luật khớp đầu tiên thì dừng. Ví dụ một lỗi hay gặp:

```
// ❌ SAI: anyRequest() đặt trước nên khớp mọi URL → luật /admin/** phía dưới không bao giờ được xét
.anyRequest().authenticated()
.requestMatchers("/admin/**").hasRole("SELLER")      // Spring báo lỗi khi khởi động

// ✅ ĐÚNG: luật cụ thể trước, luật chung sau cùng
.requestMatchers("/", "/products/**").permitAll()
.requestMatchers("/admin/**").hasRole("SELLER")
.anyRequest().authenticated()
```

Thêm một ví dụ: phải kiểm tra **quyền sở hữu dữ liệu**. `customer1` và `customer2` đều có `ROLE_CUSTOMER`, nên luật URL `/orders/**` cho cả hai đi qua. Nếu `customer2` sửa URL thành `/orders/15` (đơn của `customer1`), **luật URL không chặn được**. Controller phải tự kiểm tra `order.user.username == người đang đăng nhập` (hàm `getOwnedOrder` ở bước 3.10).

3.1.12. Theo dấu từng request qua hệ thống – 4 kịch bản

Kịch bản A – Đăng nhập (`customer1` / `123456`)

```
sequenceDiagram
    actor U as Trình duyệt
    participant F as Security Filters
    participant C as AuthController
    participant AM as AuthenticationManager
    participant UDS as CustomUserDetailsService
    participant DB as MariaDB
    participant J as JwtService
    U->>F: POST /login (username, password, _csrf)
    F->>F: CsrfFilter kiểm _csrf ✓ · /login permitAll ✓
    F->>C: chuyển tới controller
    C->>AM: authenticate(customer1, 123456)
    AM->>UDS: loadUserByUsername("customer1")
    UDS->>DB: SELECT * FROM users WHERE username='customer1'
    DB-->>UDS: password=$2a$10$uPUN..., role=ROLE_CUSTOMER, enabled=1
    UDS-->>AM: UserDetails
    AM->>AM: enabled? ✓  BCrypt.matches("123456", "$2a$10$uPUN...") ✓
    AM-->>C: Authentication [ROLE_CUSTOMER]
    C->>J: generateToken(user)
    J-->>C: eyJhbGciOi...
    C-->>U: 302 Location: /  +  Set-Cookie: ACCESS_TOKEN=eyJ...; HttpOnly
```

Kịch bản B – GUEST bấm "Thêm vào giỏ" (`POST /cart/add`, không có cookie token)

```
④ JwtAuthenticationFilter     : không có token → bỏ qua
⑤ AnonymousAuthenticationFilter: đặt Anonymous [ROLE_GUEST]
⑦ AuthorizationFilter         : /cart/** cần ROLE_CUSTOMER → ✗ AccessDeniedException
⑥ ExceptionTranslationFilter  : người dùng là Anonymous → coi là "chưa đăng nhập"
                                → 302 Location: /login                    (KHÔNG phải 403)
```

Kịch bản C – `customer1` vào trang quản trị (`GET /admin/products`, có cookie token)

```
④ JwtAuthenticationFilter     : đọc cookie → chữ ký ✓ hạn ✓ → nạp customer1 từ DB →
Authentication [ROLE_CUSTOMER]
⑤ AnonymousAuthenticationFilter: đã có Authentication → bỏ qua
⑦ AuthorizationFilter         : /admin/** cần ROLE_SELLER → ✗ AccessDeniedException
⑥ ExceptionTranslationFilter  : người dùng ĐÃ đăng nhập → thiếu quyền → forward /403
```

Kịch bản D – `seller1` xoá sản phẩm qua API (`DELETE /api/products/5`, header `Bearer`)

```
URL bắt đầu /api/ → dùng apiFilterChain (@Order(1)); không kiểm CSRF
④ JwtAuthenticationFilter     : đọc header Bearer → hợp lệ → Authentication [ROLE_SELLER]
```

```
⑦ AuthorizationFilter          : DELETE /api/** cần ROLE_SELLER ✓
ProductApiController.delete() → ProductService.delete() (@PreAuthorize SELLER ✓) → 204 No Content
```

3.1.13. Vì sao có 2 SecurityFilterChain?

Phần web và phần API có nhu cầu khác nhau (cách gửi token, CSRF, cách báo lỗi), nên ta khai báo **2 chuỗi filter**. Với mỗi request, Spring chọn **chuỗi đầu tiên có `securityMatcher` khớp**, theo thứ tự `@Order`:



3.1.14. Ánh xạ khái niệm → file code

Khái niệm	Thành phần trong eShop	Bước
Lưu người dùng, vai trò	Bảng `users`, entity `User`, enum `Role`	3.3
Đọc người dùng từ DB cho Spring	`CustomUserDetailsService`	3.4
Băm mật khẩu	Bean `BCryptPasswordEncoder`	3.7
Tạo / kiểm tra JWT	`JwtService`	3.5
Đọc JWT ở mỗi request	`JwtAuthenticationFilter`	3.6
Luật URL, GUEST, CSRF, 2 chain	`SecurityConfig`	3.7
Đăng nhập (phát cookie), đăng ký, đăng xuất	`AuthController`, `logout()`	3.8
Ẩn/hiện menu theo vai trò	`sec:authorize` trong `layout.html`	3.9
Kiểm tra quyền sở hữu đơn hàng	`OrderController.getOwnedOrder()`	3.10
Phân quyền tầng method	`@EnableMethodSecurity` + `@PreAuthorize`	3.12
API phát token	`AuthApiController`	3.13

3.1.15. Câu hỏi tự kiểm tra

1. `customer1` gõ đúng mật khẩu nhưng vào `/admin/users` → mã lỗi gì? Còn khi chưa đăng nhập?
2. Trong bảng `users` lưu `ROLE_CUSTOMER`. Viết `hasRole('ROLE_CUSTOMER')` có chạy đúng không?
3. Băm `123456` hai lần bằng BCrypt được 2 chuỗi khác nhau. Vậy lúc đăng nhập kiểm tra mật khẩu thế nào?
4. Bạn dán token của mình lên jwt.io và đọc được `"roles":"ROLE_CUSTOMER"`. Như vậy JWT có an toàn

không?

5. Vì sao chain web bật CSRF còn chain API tắt?
6. Ẩn nút "Xoá" bằng `sec:authorize` đã đủ an toàn chưa?
7. `customer2` mở `/orders/15` (đơn của `customer1`). Luật `requestMatchers("/orders/**").hasRole("CUSTOMER")` có chặn được không?

```
<details>
<summary><b>Đáp án</b></summary>
```

1. Đăng nhập rồi nhưng thiếu quyền → **403**. Chưa đăng nhập → loại **401**, web chuyển về `/login`.
2. **Không.** `hasRole` tự thêm `ROLE_` → tìm `ROLE_ROLE_CUSTOMER`. Viết `hasRole('CUSTOMER')` hoặc `hasAuthority('ROLE_CUSTOMER')`.
3. Dùng `passwordEncoder.matches(raw, hash)`: hàm này lấy salt nằm trong `hash`, băm lại `raw` với salt đó rồi so sánh.
4. Ai cũng **đọc** được payload (nên không để dữ liệu nhạy cảm), nhưng **không ai sửa** được nếu không biết khoá bí mật, vì chữ ký sẽ sai.
5. Web gửi token bằng cookie mà trình duyệt tự đính kèm, nên trang lạ có thể lợi dụng. API gửi token bằng header do client tự gắn, nên trang lạ không lợi dụng được.
6. **Chưa.** Đó chỉ là tầng giao diện; vẫn cần luật URL và `@PreAuthorize`.
7. **Không**, vì cả hai đều là CUSTOMER. Phải kiểm tra quyền sở hữu trong code (`getOwnedOrder`).

```
</details>
```

Bước 3.2 – Thêm dependency

Bổ sung vào thẻ `<dependencies>` trong `pom.xml` (đặt ngay trước thẻ `</dependencies>`):

```
```xml
        <!-- ===== Phần 3: Security + JWT ===== -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.thymeleaf.extras</groupId>
            <artifactId>thymeleaf-extras-springsecurity6</artifactId>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>0.12.6</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>0.12.6</version>
            <scope>runtime</scope>
```
```

```
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>0.12.6</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>
```

Load Maven Changes. Thêm vào `application.properties`:

```
# ===== JWT =====
# Khoa bi mat Base64, toi thieu 256 bit (32 byte). Moi nhom tu sinh khoa rieng, KHONG commit len Git that.
app.jwt.secret=N45bb7tsq8gOEJgSwlOh2QJ88qa66XO66l2+9pGjnM/YI0zwef4ob8hrFuifheOM
# Thoi han token: 1 ngay
app.jwt.expiration-ms=86400000

logging.level.org.springframework.security=DEBUG
```

Sinh khoá mới: `openssl rand -base64 48`, hoặc chạy tạm `Base64.getEncoder().encodeToString(io.jsonwebtoken.security.Keys.secretKeyFor(io.jsonwebtoken.SignatureAlgorithm.HS256).getEncoded())`.

Bước 3.3 – Entity User, Repository

`entity/Role.java`

```
package com.example.eshop.entity;

/** Vai trò lưu trong DB. ROLE_GUEST không lưu DB — gán cho người chưa đăng nhập. */
public enum Role { ROLE_SELLER, ROLE_CUSTOMER }
```

`entity/User.java`

```
package com.example.eshop.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import org.hibernate.annotations.CreationTimestamp;
import java.time.LocalDateTime;
```

```java
@Entity
@Table(name = "users")
@Getter @Setter @NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false, length = 100)
    private String password;            // chuỗi BCrypt

    @Column(name = "full_name", nullable = false, length = 100)
    private String fullName;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    private String phone;
    private String address;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 30)
    private Role role = Role.ROLE_CUSTOMER;

    private boolean enabled = true;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;
}
```

repository/UserRepository.java

```java
package com.example.eshop.repository;

import com.example.eshop.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    boolean existsByUsername(String username);
    boolean existsByEmail(String email);
}
```

Liên kết đơn hàng với người dùng – sửa `entity/Order.java`, thay comment `// (Phần 3 sẽ thêm thuộc tính user)` bằng:

```java
 @ManyToOne(fetch = FetchType.LAZY)
 @JoinColumn(name = "user_id")
    private User user;
```

và bỏ comment dòng trong `OrderRepository`:

```
List<Order> findByUserUsernameOrderByOrderDateDesc(String username);
```

Bước 3.4 – UserDetailsService: nối Spring Security với bảng `users`

`security/CustomUserDetailsService.java`

```
package com.example.eshop.security;

import com.example.eshop.entity.User;
import com.example.eshop.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User u = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("Không tìm thấy tài khoản: " +
username));

        return org.springframework.security.core.userdetails.User
                .withUsername(u.getUsername())
                .password(u.getPassword())          // BCrypt hash
                .authorities(u.getRole().name())    // "ROLE_SELLER" / "ROLE_CUSTOMER"
                .disabled(!u.isEnabled())           // tài khoản bị khoá
                .build();
    }
}
```

Role = authority có tiền tố `ROLE_`. DB lưu `ROLE_SELLER`; trong code viết `hasRole("SELLER")` (Spring tự thêm `ROLE_`) hoặc `hasAuthority("ROLE_SELLER")`.

Bước 3.5 – JwtService: tạo và kiểm tra token

`security/JwtService.java`

```
package com.example.eshop.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
```

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.stream.Collectors;

@Service
public class JwtService {

    public static final String COOKIE_NAME = "ACCESS_TOKEN";

    private final SecretKey key;
    private final long expirationMs;

    public JwtService(@Value("${app.jwt.secret}") String secret,
                      @Value("${app.jwt.expiration-ms}") long expirationMs) {
        this.key = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
        this.expirationMs = expirationMs;
    }

    public String generateToken(UserDetails user) {
        String roles = user.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority)
                .collect(Collectors.joining(","));
        Date now = new Date();
        return Jwts.builder()
                .subject(user.getUsername())
                .claim("roles", roles)
                .issuedAt(now)
                .expiration(new Date(now.getTime() + expirationMs))
                .signWith(key)                      // HS256 vì key 256+ bit
                .compact();
    }

    /** Ném JwtException nếu token sai chữ ký, sai định dạng hoặc đã hết hạn. */
    public String extractUsername(String token) throws JwtException {
        return parseClaims(token).getSubject();
    }

    private Claims parseClaims(String token) {
        return Jwts.parser()
                .verifyWith(key)
                .build()
                .parseSignedClaims(token)
                .getPayload();
    }

    public long getExpirationSeconds() {
        return expirationMs / 1000;
    }
}
```

Bước 3.6 – JwtAuthenticationFilter

`security/JwtAuthenticationFilter.java`

```java
package com.example.eshop.security;

import io.jsonwebtoken.JwtException;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

/**
 * Đọc JWT từ header "Authorization: Bearer ..." (API) hoặc cookie ACCESS_TOKEN (web),
 * hợp lệ thì đặt Authentication vào SecurityContext.
 * KHÔNG đánh dấu @Component: nếu là bean, Spring Boot sẽ đăng ký nó thêm một lần nữa
 * làm servlet filter toàn cục → chạy 2 lần cho mỗi request.
 */
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtService jwtService, UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain chain) throws ServletException, IOException {
        String token = resolveToken(request);

        if (token != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            try {
                String username = jwtService.extractUsername(token);
                // Nạp lại từ DB để tài khoản bị khoá / đổi quyền có hiệu lực ngay
                UserDetails user = userDetailsService.loadUserByUsername(username);
                if (user.isEnabled()) {
                    var auth = new UsernamePasswordAuthenticationToken(user, null,
user.getAuthorities());
                    auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContext context = SecurityContextHolder.createEmptyContext();
                    context.setAuthentication(auth);
                    SecurityContextHolder.setContext(context);
                }
            } catch (JwtException | IllegalArgumentException | UsernameNotFoundException e) {
                // Token sai/hết hạn: bỏ qua, request tiếp tục như khách (GUEST)
            }
        }
        chain.doFilter(request, response);
    }
```

```
    private String resolveToken(HttpServletRequest request) {
        String header = request.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            return header.substring(7);
        }
        if (request.getCookies() != null) {
            for (Cookie c : request.getCookies()) {
                if (JwtService.COOKIE_NAME.equals(c.getName())) {
                    return c.getValue();
                }
            }
        }
        return null;
    }
}
```

Bước 3.7 – SecurityConfig (trái tim của Phần 3)

`security/SecurityConfig.java`

```
package com.example.eshop.security;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpMethod;
import org.springframework.http.HttpStatus;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfigura
tion;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;
import org.springframework.security.web.authentication.LoginUrlAuthenticationEntryPoint;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity                       // bật @PreAuthorize
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtService jwtService;
    private final CustomUserDetailsService userDetailsService;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
```

```
    /** Spring Boot tự ráp DaoAuthenticationProvider từ UserDetailsService + PasswordEncoder. */
    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
        return config.getAuthenticationManager();
    }

    private JwtAuthenticationFilter jwtFilter() {
        return new JwtAuthenticationFilter(jwtService, userDetailsService);
    }

    // ================= CHAIN 1: REST API (/api/**) =================
    @Bean
    @Order(1)
    public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**")
            .csrf(AbstractHttpConfigurer::disable)                // API dùng header Bearer, không
dùng cookie
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .anonymous(a -> a.authorities("ROLE_GUEST"))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
                .requestMatchers("/api/orders/**").hasRole("CUSTOMER")
                .anyRequest().hasRole("SELLER"))
            .exceptionHandling(e -> e
                .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))  //
401
            .addFilterBefore(jwtFilter(), UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    // ================= CHAIN 2: WEB (Thymeleaf) =================
    @Bean
    @Order(2)
    public SecurityFilterChain webFilterChain(HttpSecurity http) throws Exception {
        http
            // Spring Security không lưu đăng nhập vào session — thông tin đăng nhập nằm trong JWT
cookie.
            // (Ứng dụng vẫn dùng HttpSession cho giỏ hàng và CSRF token.)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            // Người chưa đăng nhập mang quyền ROLE_GUEST
            .anonymous(a -> a.authorities("ROLE_GUEST"))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/css/**", "/js/**", "/images/**", "/favicon.ico", "/error",
"/403").permitAll()
                .requestMatchers("/", "/products", "/products/**", "/login").permitAll()
                .requestMatchers("/register").hasRole("GUEST")
                .requestMatchers("/admin/**").hasRole("SELLER")
                .requestMatchers("/cart/**", "/checkout/**", "/orders/**").hasRole("CUSTOMER")
                .anyRequest().authenticated())
            .exceptionHandling(e -> e
                .authenticationEntryPoint(new LoginUrlAuthenticationEntryPoint("/login"))  // chưa
login → /login
                .accessDeniedPage("/403"))                                                  //
thiếu quyền → /403
            .logout(l -> l
                .logoutUrl("/logout")                                  // POST /logout (có CSRF)
                .deleteCookies(JwtService.COOKIE_NAME, "JSESSIONID")
```

```
                .invalidateHttpSession(true)                        // xoá luôn giỏ hàng
                .logoutSuccessUrl("/?logout"))
            .addFilterBefore(jwtFilter(), UsernamePasswordAuthenticationFilter.class);
            // CSRF để mặc định (bật): th:action của Thymeleaf tự chèn input _csrf
        return http.build();
    }
}
```

Giải thích các điểm quan trọng:

1. **Thứ tự luật** trong `authorizeHttpRequests` xét từ trên xuống, gặp luật khớp đầu tiên là dừng → luật cụ thể đặt trước, `anyRequest()` luôn cuối.
2. `/products/**` là `permitAll` nhưng `/admin/products/**` là `SELLER` – hai mẫu khác nhau vì mẫu luôn khớp từ đầu đường dẫn.
3. Không dùng `formLogin()`: ta tự viết `POST /login` trong controller để **phát JWT** thay vì tạo session.
4. Khách (GUEST) bấm *Thêm vào giỏ* → `AccessDeniedException` với người dùng *anonymous* → `ExceptionTranslationFilter` coi là "chưa đăng nhập" → chuyển tới `/login` (không phải `/403`).
5. CUSTOMER vào `/admin/...` hoặc `/register` → `/403`.
6. Cookie JWT là **HttpOnly** (JavaScript không đọc được → chống đánh cắp qua XSS) và `SameSite=Lax`; CSRF token vẫn bật vì trình duyệt tự gửi cookie.

Bước 3.8 – Đăng nhập, đăng ký, đăng xuất

`dto/RegisterForm.java`

```
package com.example.eshop.dto;

import jakarta.validation.constraints.*;
import lombok.Getter;
import lombok.Setter;

@Getter @Setter
public class RegisterForm {

    @NotBlank(message = "Nhập tên đăng nhập")
    @Size(min = 4, max = 50, message = "Từ 4 đến 50 ký tự")
    @Pattern(regexp = "^[a-zA-Z0-9_]+$", message = "Chỉ gồm chữ không dấu, số, gạch dưới")
    private String username;

    @NotBlank(message = "Nhập mật khẩu")
    @Size(min = 6, max = 50, message = "Mật khẩu từ 6 ký tự")
    private String password;

    @NotBlank(message = "Nhập lại mật khẩu")
    private String confirmPassword;

    @NotBlank(message = "Nhập họ tên")
    private String fullName;

    @NotBlank(message = "Nhập email")
    @Email(message = "Email không hợp lệ")
    private String email;

    @Pattern(regexp = "^$|^0\\d{9}$", message = "SĐT gồm 10 số, bắt đầu bằng 0")
```

```
    private String phone;

    private String address;
}
```

service/UserService.java

```
package com.example.eshop.service;

import com.example.eshop.dto.RegisterForm;
import com.example.eshop.dto.UserForm;
import com.example.eshop.entity.Role;
import com.example.eshop.entity.User;
import com.example.eshop.exception.NotFoundException;
import com.example.eshop.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.util.StringUtils;

import java.util.List;

@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public User findByUsername(String username) {
        return userRepository.findByUsername(username)
                .orElseThrow(() -> new NotFoundException("Không tìm thấy tài khoản " + username));
    }

    public User findById(Long id) {
        return userRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("Không tìm thấy tài khoản id = " + id));
    }

    public List<User> findAll() {
        return userRepository.findAll();
    }

    /** GUEST tự đăng ký → luôn là CUSTOMER. */
    @Transactional
    public User register(RegisterForm f) {
        if (userRepository.existsByUsername(f.getUsername()))
            throw new IllegalArgumentException("Tên đăng nhập đã tồn tại");
        if (userRepository.existsByEmail(f.getEmail()))
            throw new IllegalArgumentException("Email đã được sử dụng");

        User u = new User();
        u.setUsername(f.getUsername());
        u.setPassword(passwordEncoder.encode(f.getPassword()));     // KHÔNG lưu mật khẩu thô
        u.setFullName(f.getFullName());
        u.setEmail(f.getEmail());
        u.setPhone(f.getPhone());
        u.setAddress(f.getAddress());
```

```
        u.setRole(Role.ROLE_CUSTOMER);
        u.setEnabled(true);
        return userRepository.save(u);
    }

    /** SELLER tạo / sửa tài khoản bất kỳ. Để trống mật khẩu khi sửa = giữ mật khẩu cũ. */
    @Transactional
    public User saveByAdmin(UserForm f) {
        User u;
        if (f.getId() == null) {
            if (!StringUtils.hasText(f.getPassword()))
                throw new IllegalArgumentException("Nhập mật khẩu cho tài khoản mới");
            if (userRepository.existsByUsername(f.getUsername()))
                throw new IllegalArgumentException("Tên đăng nhập đã tồn tại");
            u = new User();
            u.setUsername(f.getUsername());
        } else {
            u = findById(f.getId());
        }
        u.setFullName(f.getFullName());
        u.setEmail(f.getEmail());
        u.setPhone(f.getPhone());
        u.setAddress(f.getAddress());
        u.setRole(f.getRole());
        u.setEnabled(f.isEnabled());
        if (StringUtils.hasText(f.getPassword())) {
            u.setPassword(passwordEncoder.encode(f.getPassword()));
        }
        return userRepository.save(u);
    }

    @Transactional
    public void delete(Long id, String currentUsername) {
        User u = findById(id);
        if (u.getUsername().equals(currentUsername))
            throw new IllegalStateException("Không thể tự xoá tài khoản đang đăng nhập");
        userRepository.delete(u);        // orders.user_id → NULL (ON DELETE SET NULL)
    }
}
```

controller/AuthController.java

```
package com.example.eshop.controller;

import com.example.eshop.dto.RegisterForm;
import com.example.eshop.security.JwtService;
import com.example.eshop.service.UserService;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpHeaders;
import org.springframework.http.ResponseCookie;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.DisabledException;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
```

```
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequiredArgsConstructor
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;
    private final UserService userService;

    @GetMapping("/login")
    public String loginPage() {
        return "auth/login";
    }

    @PostMapping("/login")
    public String login(@RequestParam String username, @RequestParam String password,
                        HttpServletResponse response) {
        try {
            Authentication auth = authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(username, password));
            UserDetails user = (UserDetails) auth.getPrincipal();

            ResponseCookie cookie = ResponseCookie.from(JwtService.COOKIE_NAME,
jwtService.generateToken(user))
                    .httpOnly(true)
                    .path("/")
                    .maxAge(jwtService.getExpirationSeconds())
                    .sameSite("Lax")
                    // .secure(true)   // bật khi chạy HTTPS
                    .build();
            response.addHeader(HttpHeaders.SET_COOKIE, cookie.toString());

            boolean isSeller = user.getAuthorities().stream()
                    .anyMatch(a -> a.getAuthority().equals("ROLE_SELLER"));
            return isSeller ? "redirect:/admin/products" : "redirect:/";
        } catch (DisabledException e) {
            return "redirect:/login?error=disabled";
        } catch (AuthenticationException e) {
            return "redirect:/login?error=bad";
        }
    }

    @GetMapping("/register")
    public String registerForm(Model model) {
        model.addAttribute("form", new RegisterForm());
        return "auth/register";
    }

    @PostMapping("/register")
    public String register(@Valid @ModelAttribute("form") RegisterForm form, BindingResult result,
                           RedirectAttributes ra) {
        if (!form.getPassword().equals(form.getConfirmPassword())) {
            result.rejectValue("confirmPassword", "mismatch", "Mật khẩu nhập lại không khớp");
        }
        if (result.hasErrors()) {
            return "auth/register";
        }
```

```
        try {
            userService.register(form);
        } catch (IllegalArgumentException e) {
            result.reject("register", e.getMessage());
            return "auth/register";
        }
        ra.addFlashAttribute("message", "Đăng ký thành công, mời bạn đăng nhập");
        return "redirect:/login";
    }
}
```

controller/ErrorPageController.java

```
package com.example.eshop.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class ErrorPageController {
    @RequestMapping("/403")          // nhận cả GET và POST (accessDeniedPage forward nguyên
method)
    public String accessDenied() {
        return "error/403";
    }
}
```

templates/auth/login.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Đăng nhập')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container" style="max-width:420px">
    <div th:replace="~{fragments/layout :: alerts}"></div>
    <h3 class="mb-3">Đăng nhập</h3>

    <div th:if="${param.error != null and param.error[0] == 'disabled'}" class="alert alert-
warning">Tài khoản đã bị khoá.</div>
    <div th:if="${param.error != null and param.error[0] == 'bad'}" class="alert alert-danger">Sai
tên đăng nhập hoặc mật khẩu.</div>

    <!-- th:action tự chèn <input type="hidden" name="_csrf" ...> -->
    <form th:action="@{/login}" method="post">
        <div class="mb-3">
            <label class="form-label">Tên đăng nhập</label>
            <input class="form-control" name="username" required autofocus>
        </div>
        <div class="mb-3">
            <label class="form-label">Mật khẩu</label>
            <input class="form-control" type="password" name="password" required>
        </div>
        <button class="btn btn-primary w-100">Đăng nhập</button>
    </form>
    <p class="mt-3 text-center">Chưa có tài khoản? <a th:href="@{/register}">Đăng ký</a></p>
    <p class="small text-muted">Demo: seller1 / customer1 – mật khẩu 123456</p>
```

```
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

templates/auth/register.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Đăng ký')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container" style="max-width:560px">
    <h3 class="mb-3">Đăng ký tài khoản khách hàng</h3>
    <form th:action="@{/register}" th:object="${form}" method="post">
        <div th:if="${#fields.hasGlobalErrors()}" class="alert alert-danger"
th:errors="*{global}"></div>

        <div class="mb-2">
            <label class="form-label">Tên đăng nhập *</label>
            <input class="form-control" th:field="*{username}" th:errorclass="is-invalid">
            <div class="invalid-feedback" th:errors="*{username}"></div>
        </div>
        <div class="mb-2">
            <label class="form-label">Họ tên *</label>
            <input class="form-control" th:field="*{fullName}" th:errorclass="is-invalid">
            <div class="invalid-feedback" th:errors="*{fullName}"></div>
        </div>
        <div class="mb-2">
            <label class="form-label">Email *</label>
            <input type="email" class="form-control" th:field="*{email}" th:errorclass="is-
invalid">
            <div class="invalid-feedback" th:errors="*{email}"></div>
        </div>
        <div class="row">
            <div class="col mb-2">
                <label class="form-label">Số điện thoại</label>
                <input class="form-control" th:field="*{phone}" th:errorclass="is-invalid">
                <div class="invalid-feedback" th:errors="*{phone}"></div>
            </div>
            <div class="col mb-2">
                <label class="form-label">Địa chỉ</label>
                <input class="form-control" th:field="*{address}">
            </div>
        </div>
        <div class="mb-2">
            <label class="form-label">Mật khẩu *</label>
            <input type="password" class="form-control" th:field="*{password}" th:errorclass="is-
invalid">
            <div class="invalid-feedback" th:errors="*{password}"></div>
        </div>
        <div class="mb-3">
            <label class="form-label">Nhập lại mật khẩu *</label>
            <input type="password" class="form-control" th:field="*{confirmPassword}"
th:errorclass="is-invalid">
            <div class="invalid-feedback" th:errors="*{confirmPassword}"></div>
        </div>
        <button class="btn btn-success w-100">Đăng ký</button>
    </form>
```

```
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

templates/error/403.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Không có quyền')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container text-center py-5">
    <h1 class="display-4">403</h1>
    <p class="lead">Bạn không có quyền truy cập chức năng này.</p>
    <a class="btn btn-primary" th:href="@{/}">Về trang chủ</a>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

Bước 3.9 – Navbar theo vai trò (sec:authorize)

Thay thẻ <html> của fragments/layout.html thành:

```
<html xmlns:th="http://www.thymeleaf.org" xmlns:sec="http://www.thymeleaf.org/extras/spring-
security">
```

và thay toàn bộ fragment navbar bằng **phiên bản cuối**:

```
<nav th:fragment="navbar" class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
    <div class="container">
        <a class="navbar-brand fw-bold" th:href="@{/}">🛒 eShop</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#mainNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="mainNav">
            <ul class="navbar-nav me-auto">
                <li class="nav-item"><a class="nav-link" th:href="@{/products}">Sản phẩm</a></li>

                <!-- Chỉ SELLER -->
                <li class="nav-item dropdown" sec:authorize="hasRole('SELLER')">
                    <a class="nav-link dropdown-toggle" href="#" data-bs-toggle="dropdown">Quản
trị</a>
                    <ul class="dropdown-menu">
                        <li><a class="dropdown-item" th:href="@{/admin/categories}">Danh
mục</a></li>
                        <li><a class="dropdown-item" th:href="@{/admin/products}">Sản phẩm</a></li>
                        <li><a class="dropdown-item" th:href="@{/admin/orders}">Đơn hàng</a></li>
                        <li><a class="dropdown-item" th:href="@{/admin/users}">Tài khoản</a></li>
                    </ul>
                </li>

                <!-- Chỉ CUSTOMER -->
```

```
                <li class="nav-item" sec:authorize="hasRole('CUSTOMER')">
                    <a class="nav-link" th:href="@{/orders/my}">Đơn hàng của tôi</a>
                </li>
            </ul>

            <form class="d-flex" th:action="@{/products}" method="get">
                <input class="form-control me-2" name="keyword" placeholder="Tìm sản phẩm..."
th:value="${keyword}">
                <button class="btn btn-light" type="submit">Tìm</button>
            </form>

            <ul class="navbar-nav ms-3">
                <!-- GUEST và CUSTOMER thấy giỏ hàng (GUEST bấm vào sẽ được mời đăng nhập) -->
                <li class="nav-item" sec:authorize="!hasRole('SELLER')">
                    <a class="nav-link" th:href="@{/cart}">Giỏ hàng
                        <span class="badge bg-warning text-dark"
th:text="${cartCount}">0</span></a>
                </li>

                <!-- Chỉ GUEST -->
                <li class="nav-item" sec:authorize="hasRole('GUEST')"><a class="nav-link"
th:href="@{/login}">Đăng nhập</a></li>
                <li class="nav-item" sec:authorize="hasRole('GUEST')"><a class="nav-link"
th:href="@{/register}">Đăng ký</a></li>

                <!-- Đã đăng nhập -->
                <li class="nav-item dropdown" sec:authorize="isAuthenticated()">
                    <a class="nav-link dropdown-toggle" href="#" data-bs-toggle="dropdown">
                        👤 <span sec:authentication="name"></span>
                    </a>
                    <ul class="dropdown-menu dropdown-menu-end">
                        <li><span class="dropdown-item-text small text-muted"
sec:authentication="principal.authorities"></span></li>
                        <li>
                            <!-- Logout phải là POST để có CSRF token -->
                            <form th:action="@{/logout}" method="post">
                                <button class="dropdown-item">Đăng xuất</button>
                            </form>
                        </li>
                    </ul>
                </li>
            </ul>
        </div>
    </div>
</nav>
```

Ẩn nút *Thêm vào giỏ* với SELLER trong `shop/index.html` và `shop/detail.html` – thêm thuộc tính `sec:authorize="!hasRole('SELLER')"` vào thẻ `<form th:action="@{/cart/add}" ...>` (và thêm `xmlns:sec` vào thẻ `<html>` của 2 file đó).

`sec:authorize` **chỉ ẩn giao diện**, không phải bảo mật thật. Bảo mật thật nằm ở `SecurityConfig` (URL) và `@PreAuthorize` (method) – thử gõ thẳng URL `/admin/products` khi là CUSTOMER để thấy.

Bước 3.10 – Gắn đơn hàng với khách hàng đăng nhập

`OrderService` – sửa chữ ký `placeOrder` để nhận người đặt:

```
private final UserRepository userRepository;   // thêm field (import
com.example.eshop.repository.UserRepository)

    @Transactional
    public Order placeOrder(CheckoutForm form, Cart cart, String username) {
        // ... giữ nguyên phần thân như Phần 2, thêm 1 dòng sau khi tạo order:
        order.setUser(userRepository.findByUsername(username).orElse(null));
        // ...
    }

    public List<Order> findByUsername(String username) {
        return orderRepository.findByUserUsernameOrderByOrderDateDesc(username);
    }
```

`CheckoutController` – điền sẵn thông tin khách và truyền username:

```
// import com.example.eshop.entity.User; import com.example.eshop.service.UserService; import
java.security.Principal;
    private final UserService userService;        // thêm field

    @GetMapping
    public String form(Model model, Principal principal) {        // java.security.Principal
        if (cart.isEmpty()) return "redirect:/cart";
        User u = userService.findByUsername(principal.getName());
        CheckoutForm f = new CheckoutForm();
        f.setCustomerName(u.getFullName());
        f.setPhone(u.getPhone());
        f.setAddress(u.getAddress());
        model.addAttribute("form", f);
        model.addAttribute("cart", cart);
        return "checkout/form";
    }

    @PostMapping
    public String submit(@Valid @ModelAttribute("form") CheckoutForm form, BindingResult result,
                         Model model, RedirectAttributes ra, Principal principal) {
        // ... như cũ, chỉ đổi dòng gọi service:
        Order order = orderService.placeOrder(form, cart, principal.getName());
        // ...
    }
```

`OrderController` – thêm "Đơn hàng của tôi" và **kiểm tra quyền sở hữu** (CUSTOMER A không được xem đơn của B chỉ bằng cách đổi id trên URL):

```
package com.example.eshop.controller;

import com.example.eshop.entity.Order;
import com.example.eshop.service.OrderService;
import lombok.RequiredArgsConstructor;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

import java.security.Principal;
```

```
@Controller
@RequestMapping("/orders")
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;

    @GetMapping("/my")
    public String myOrders(Principal principal, Model model) {
        model.addAttribute("orders", orderService.findByUsername(principal.getName()));
        return "order/my-orders";
    }

    @GetMapping("/{id}")
    public String detail(@PathVariable Long id, Principal principal, Model model) {
        model.addAttribute("order", getOwnedOrder(id, principal));
        return "order/detail";
    }

    @GetMapping("/{id}/payment")
    public String payment(@PathVariable Long id, Principal principal, Model model) {
        model.addAttribute("order", getOwnedOrder(id, principal));
        return "order/payment";
    }

    @PostMapping("/{id}/pay")
    public String pay(@PathVariable Long id, Principal principal, RedirectAttributes ra) {
        getOwnedOrder(id, principal);
        orderService.markPaid(id);
        ra.addFlashAttribute("message", "Thanh toán thành công đơn #" + id);
        return "redirect:/orders/" + id;
    }

    private Order getOwnedOrder(Long id, Principal principal) {
        Order o = orderService.findById(id);
        if (o.getUser() == null || !o.getUser().getUsername().equals(principal.getName())) {
            throw new AccessDeniedException("Đơn hàng không thuộc về bạn");   // → trang /403
        }
        return o;
    }
}
```

templates/order/my-orders.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Đơn hàng của tôi')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <h3>Đơn hàng của tôi</h3>
    <div th:if="${#lists.isEmpty(orders)}" class="alert alert-info">Bạn chưa có đơn hàng nào.</div>
    <table class="table table-hover" th:unless="${#lists.isEmpty(orders)}">
        <thead><tr><th>#</th><th>Ngày đặt</th><th>Thanh toán</th><th>Trạng thái</th><th
class="text-end">Tổng tiền</th><th></th></tr></thead>
        <tbody>
        <tr th:each="o : ${orders}">
            <td th:text="${o.id}"></td>
            <td th:text="${#temporals.format(o.orderDate, 'dd/MM/yyyy HH:mm')}"></td>
```

```
            <td th:text="${o.paymentMethod}"></td>
            <td><span class="badge bg-info text-dark" th:text="${o.status}"></span></td>
            <td class="text-end" th:text="${#numbers.formatDecimal(o.totalAmount, 0, 'POINT', 0,
'COMMA')} + ' đ'"></td>
            <td><a class="btn btn-sm btn-outline-primary"
th:href="@{/orders/{id}(id=${o.id})}">Xem</a></td>
        </tr>
        </tbody>
    </table>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

Bước 3.11 – SELLER: CRUD bảng `users`

`dto/UserForm.java` – dùng DTO để **không bao giờ** đưa chuỗi hash mật khẩu ra form

```
package com.example.eshop.dto;

import com.example.eshop.entity.Role;
import com.example.eshop.entity.User;
import jakarta.validation.constraints.*;
import lombok.Getter;
import lombok.Setter;

@Getter @Setter
public class UserForm {
    private Long id;

    @NotBlank(message = "Nhập tên đăng nhập")
    @Size(min = 4, max = 50)
    private String username;

    @Size(max = 50)
    private String password;             // để trống khi sửa = giữ nguyên

    @NotBlank(message = "Nhập họ tên")
    private String fullName;

    @NotBlank @Email(message = "Email không hợp lệ")
    private String email;

    private String phone;
    private String address;

    @NotNull
    private Role role = Role.ROLE_CUSTOMER;

    private boolean enabled = true;

    public static UserForm from(User u) {
        UserForm f = new UserForm();
        f.setId(u.getId());
        f.setUsername(u.getUsername());
        f.setFullName(u.getFullName());
        f.setEmail(u.getEmail());
        f.setPhone(u.getPhone());
```

```
        f.setAddress(u.getAddress());
        f.setRole(u.getRole());
        f.setEnabled(u.isEnabled());
        return f;
    }
}
```

controller/admin/AdminUserController.java

```
package com.example.eshop.controller.admin;

import com.example.eshop.dto.UserForm;
import com.example.eshop.entity.Role;
import com.example.eshop.service.UserService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

import java.security.Principal;

@Controller
@RequestMapping("/admin/users")
@RequiredArgsConstructor
public class AdminUserController {

    private final UserService userService;

    @GetMapping
    public String list(Model model) {
        model.addAttribute("users", userService.findAll());
        return "admin/user/list";
    }

    @GetMapping("/new")
    public String createForm(Model model) {
        return showForm(model, new UserForm());
    }

    @GetMapping("/{id}/edit")
    public String editForm(@PathVariable Long id, Model model) {
        return showForm(model, UserForm.from(userService.findById(id)));
    }

    @PostMapping("/save")
    public String save(@Valid @ModelAttribute("userForm") UserForm form, BindingResult result,
                       Model model, RedirectAttributes ra) {
        if (result.hasErrors()) return showForm(model, form);
        try {
            userService.saveByAdmin(form);
        } catch (IllegalArgumentException e) {
            result.reject("save", e.getMessage());
            return showForm(model, form);
        } catch (DataIntegrityViolationException e) {
            result.reject("save", "Email đã được sử dụng");
```

```
            return showForm(model, form);
        }
        ra.addFlashAttribute("message", "Đã lưu tài khoản " + form.getUsername());
        return "redirect:/admin/users";
    }

    @PostMapping("/{id}/delete")
    public String delete(@PathVariable Long id, Principal principal, RedirectAttributes ra) {
        try {
            userService.delete(id, principal.getName());
            ra.addFlashAttribute("message", "Đã xoá tài khoản");
        } catch (IllegalStateException e) {
            ra.addFlashAttribute("error", e.getMessage());
        }
        return "redirect:/admin/users";
    }

    private String showForm(Model model, UserForm form) {
        model.addAttribute("userForm", form);
        model.addAttribute("roles", Role.values());
        return "admin/user/form";
    }
}
```

templates/admin/user/list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Tài khoản')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: alerts}"></div>
    <div class="d-flex justify-content-between align-items-center mb-3">
        <h3>Tài khoản</h3>
        <a class="btn btn-success" th:href="@{/admin/users/new}">+ Thêm tài khoản</a>
    </div>
    <table class="table table-bordered align-middle">
        <thead class="table-light"><tr><th>ID</th><th>Username</th><th>Họ
tên</th><th>Email</th><th>Vai trò</th><th>Trạng thái</th><th></th></tr></thead>
        <tbody>
        <tr th:each="u : ${users}">
            <td th:text="${u.id}"></td>
            <td th:text="${u.username}"></td>
            <td th:text="${u.fullName}"></td>
            <td th:text="${u.email}"></td>
            <td><span class="badge" th:classappend="${u.role.name() == 'ROLE_SELLER'} ? 'bg-
danger' : 'bg-primary'" th:text="${u.role}"></span></td>
            <td th:text="${u.enabled} ? 'Hoạt động' : 'Bị khoá'"></td>
            <td class="text-nowrap">
                <a class="btn btn-sm btn-outline-primary"
th:href="@{/admin/users/{id}/edit(id=${u.id})}">Sửa</a>
                <form th:action="@{/admin/users/{id}/delete(id=${u.id})}" method="post" class="d-
inline"
                      onsubmit="return confirm('Xoá tài khoản này?')">
                    <button class="btn btn-sm btn-outline-danger">Xoá</button>
                </form>
            </td>
        </tr>
```

```
        </tbody>
    </table>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

templates/admin/user/form.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Tài khoản')}"></head>
<body>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<main class="container" style="max-width:640px">
    <h3 th:text="${userForm.id == null} ? 'Thêm tài khoản' : 'Sửa tài khoản'"></h3>
    <form th:action="@{/admin/users/save}" th:object="${userForm}" method="post">
        <div th:if="${#fields.hasGlobalErrors()}" class="alert alert-danger"
th:errors="*{global}"></div>
        <input type="hidden" th:field="*{id}">

        <div class="mb-2">
            <label class="form-label">Tên đăng nhập *</label>
            <input class="form-control" th:field="*{username}" th:readonly="*{id != null}"
th:errorclass="is-invalid">
            <div class="invalid-feedback" th:errors="*{username}"></div>
        </div>
        <div class="mb-2">
            <label class="form-label">Mật khẩu <small class="text-muted" th:if="*{id != null}">(để
trống nếu không đổi)</small></label>
            <input type="password" class="form-control" th:field="*{password}">
        </div>
        <div class="mb-2">
            <label class="form-label">Họ tên *</label>
            <input class="form-control" th:field="*{fullName}" th:errorclass="is-invalid">
            <div class="invalid-feedback" th:errors="*{fullName}"></div>
        </div>
        <div class="mb-2">
            <label class="form-label">Email *</label>
            <input class="form-control" th:field="*{email}" th:errorclass="is-invalid">
            <div class="invalid-feedback" th:errors="*{email}"></div>
        </div>
        <div class="row">
            <div class="col mb-2"><label class="form-label">SĐT</label><input class="form-control"
th:field="*{phone}"></div>
            <div class="col mb-2">
                <label class="form-label">Vai trò</label>
                <select class="form-select" th:field="*{role}">
                    <option th:each="r : ${roles}" th:value="${r}" th:text="${r}"></option>
                </select>
            </div>
        </div>
        <div class="mb-2"><label class="form-label">Địa chỉ</label><input class="form-control"
th:field="*{address}"></div>
        <div class="form-check mb-3">
            <input class="form-check-input" type="checkbox" th:field="*{enabled}" id="en">
            <label class="form-check-label" for="en">Đang hoạt động (bỏ tick = khoá tài
khoản)</label>
        </div>
```

```
        <button class="btn btn-primary">Lưu</button>
        <a class="btn btn-secondary" th:href="@{/admin/users}">Huỷ</a>
    </form>
</main>
<footer th:replace="~{fragments/layout :: footer}"></footer>
</body>
</html>
```

Bước 3.12 – Lớp bảo vệ thứ hai: Method Security

Thêm `@PreAuthorize` vào các phương thức **ghi** của service – dù ai đó quên khai báo URL trong `SecurityConfig` (hoặc gọi qua REST API) thì service vẫn chặn:

```
import org.springframework.security.access.prepost.PreAuthorize;

// CategoryService
@PreAuthorize("hasRole('SELLER')") public Category save(Category category) { ... }
@PreAuthorize("hasRole('SELLER')") public void delete(Long id) { ... }

// ProductService
@PreAuthorize("hasRole('SELLER')") @Transactional public Product save(Product product, Long
categoryId) { ... }
@PreAuthorize("hasRole('SELLER')") public void delete(Long id) { ... }

// OrderService
@PreAuthorize("hasRole('CUSTOMER')") @Transactional public Order placeOrder(...) { ... }
@PreAuthorize("hasRole('SELLER')")   @Transactional public void updateStatus(...) { ... }
@PreAuthorize("hasRole('SELLER')")   @Transactional public void delete(Long id) { ... }

// UserService
@PreAuthorize("hasRole('SELLER')") @Transactional public User saveByAdmin(UserForm f) { ... }
@PreAuthorize("hasRole('SELLER')") @Transactional public void delete(Long id, String
currentUsername) { ... }
```

Tầng	Cách viết	Vai trò
URL	`requestMatchers("/admin/**").hasRole("SELLER")`	Chặn sớm ở filter
Method	`@PreAuthorize("hasRole('SELLER')")`	Bảo vệ nghiệp vụ, độc lập với URL
View	`sec:authorize="hasRole('SELLER')"`	Chỉ ẩn/hiện giao diện

Bước 3.13 – REST API với JWT Bearer

`api/AuthApiController.java`

```
package com.example.eshop.api;

import com.example.eshop.security.JwtService;
```

```
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthApiController {

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;

    public record LoginRequest(String username, String password) {}
    public record TokenResponse(String accessToken, String tokenType, long expiresIn) {}

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {
        try {
            Authentication auth = authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(req.username(), req.password()));
            String token = jwtService.generateToken((UserDetails) auth.getPrincipal());
            return ResponseEntity.ok(new TokenResponse(token, "Bearer",
jwtService.getExpirationSeconds()));
        } catch (AuthenticationException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                    .body(Map.of("error", "Sai tài khoản, mật khẩu hoặc tài khoản bị khoá"));
        }
    }
}
```

api/ProductApiController.java

```
package com.example.eshop.api;

import com.example.eshop.entity.Product;
import com.example.eshop.service.ProductService;
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.math.BigDecimal;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductApiController {

    private final ProductService productService;
```

```
    /** DTO trả ra JSON — không trả entity trực tiếp (tránh lỗi proxy LAZY, vòng lặp JSON). */
    public record ProductDto(Long id, String name, String description, BigDecimal price,
                             Integer quantity, String imageUrl, Long categoryId, String
categoryName) {
        static ProductDto from(Product p) {
            return new ProductDto(p.getId(), p.getName(), p.getDescription(), p.getPrice(),
                    p.getQuantity(), p.getImageUrl(), p.getCategory().getId(),
p.getCategory().getName());
        }
    }

    public record ProductRequest(@NotBlank String name, String description,
                                 @NotNull @DecimalMin("0") BigDecimal price,
                                 @NotNull @Min(0) Integer quantity,
                                 String imageUrl, @NotNull Long categoryId) {}

    @GetMapping                                              // GUEST, CUSTOMER, SELLER
    public Page<ProductDto> list(@RequestParam(required = false) String keyword,
                                 @RequestParam(required = false) Long categoryId,
                                 @RequestParam(defaultValue = "0") int page,
                                 @RequestParam(defaultValue = "10") int size) {
        return productService.search(keyword, categoryId, page, size).map(ProductDto::from);
    }

    @GetMapping("/{id}")
    public ProductDto get(@PathVariable Long id) {
        return ProductDto.from(productService.findById(id));
    }

    @PostMapping                                             // SELLER
    @ResponseStatus(HttpStatus.CREATED)
    public ProductDto create(@Valid @RequestBody ProductRequest req) {
        Product p = new Product();
        apply(req, p);
        return ProductDto.from(productService.save(p, req.categoryId()));
    }

    @PutMapping("/{id}")                                     // SELLER
    public ProductDto update(@PathVariable Long id, @Valid @RequestBody ProductRequest req) {
        Product p = productService.findById(id);
        apply(req, p);
        return ProductDto.from(productService.save(p, req.categoryId()));
    }

    @DeleteMapping("/{id}")                                  // SELLER
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        productService.delete(id);
    }

    private void apply(ProductRequest r, Product p) {
        p.setName(r.name());
        p.setDescription(r.description());
        p.setPrice(r.price());
        p.setQuantity(r.quantity());
        p.setImageUrl(r.imageUrl());
    }
}
```

`api/ApiExceptionHandler.java` – trả lỗi dạng JSON cho API

```
package com.example.eshop.api;

import com.example.eshop.exception.NotFoundException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.Map;

@RestControllerAdvice(basePackages = "com.example.eshop.api")
public class ApiExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<Map<String, String>> notFound(NotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("error", e.getMessage()));
    }

    @ExceptionHandler({IllegalStateException.class, IllegalArgumentException.class})
    public ResponseEntity<Map<String, String>> badRequest(RuntimeException e) {
        return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> invalid(MethodArgumentNotValidException e) {
        String msg = e.getBindingResult().getFieldErrors().stream()
                .map(f -> f.getField() + ": " + f.getDefaultMessage())
                .reduce((a, b) -> a + "; " + b).orElse("Dữ liệu không hợp lệ");
        return ResponseEntity.badRequest().body(Map.of("error", msg));
    }
}
```

Không bắt `AccessDeniedException` ở đây: để nó lan ra `ExceptionTranslationFilter` trả **403** đúng chuẩn.

Kiểm thử tổng hợp

Khởi động lại ứng dụng. Mở **F12 → Application → Cookies** để quan sát cookie `ACCESS_TOKEN`.

Trên trình duyệt

#	Vai trò	Thao tác	Kết quả mong đợi
1	GUEST	Mở `/`, tìm "laptop", lọc danh mục, xem chi tiết	Được phép; navbar có *Đăng nhập / Đăng ký*
2	GUEST	Bấm *Thêm vào giỏ* hoặc vào `/cart`	Chuyển tới `/login`
3	GUEST	Vào `/admin/products`	Chuyển tới `/login`

#	Vai trò	Thao tác	Kết quả mong đợi
4	GUEST	Đăng ký tài khoản `sv01`	Thành công; bảng `users` có `sv01`, `password` dạng `$2a$10$...`, `role = ROLE_CUSTOMER`
5	–	Đăng nhập `customer1` / sai mật khẩu	*Sai tên đăng nhập hoặc mật khẩu*
6	–	Đăng nhập `locked` / `123456`	*Tài khoản đã bị khoá*
7	CUSTOMER	Đăng nhập `customer1`	Về trang chủ; có cookie `ACCESS_TOKEN` (HttpOnly ✓); menu *Đơn hàng của tôi*
8	CUSTOMER	Mua 2 sản phẩm, thanh toán	Form điền sẵn họ tên/SĐT/địa chỉ; đơn có `user_id` của customer1
9	CUSTOMER	Gõ `/admin/products` hoặc `/register`	Trang **403**
10	CUSTOMER	Đăng nhập customer2 (trình duyệt khác), mở `/orders/{id}` đơn của customer1	Trang **403**
11	SELLER	Đăng nhập `seller1`	Chuyển tới `/admin/products`; menu *Quản trị* đủ 4 mục; không có giỏ hàng
12	SELLER	CRUD danh mục, sản phẩm; đổi trạng thái đơn; khoá tài khoản `customer2`	Thành công; customer2 đang đăng nhập sẽ thành GUEST ở request kế tiếp
13	SELLER	Xoá chính tài khoản `seller1`	Báo lỗi *Không thể tự xoá*
14	Bất kỳ	Bấm *Đăng xuất*	Cookie `ACCESS_TOKEN` bị xoá, giỏ hàng bị xoá, về GUEST
15	Bất kỳ	F12 → xem mã nguồn một form POST	Có `<input type="hidden" name="_csrf" ...>`

REST API (Postman hoặc curl – Windows dùng `curl.exe`)

```
# 1. GUEST xem sản phẩm: 200
curl -i "http://localhost:8080/api/products?keyword=iphone"

# 2. GUEST thêm sản phẩm: 401
curl -i -X POST http://localhost:8080/api/products -H "Content-Type: application/json" -d "{}"

# 3. Lấy token SELLER
curl -s -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" \
     -d "{\"username\":\"seller1\",\"password\":\"123456\"}"
# → {"accessToken":"eyJhbGciOiJIUzI1NiJ9....","tokenType":"Bearer","expiresIn":86400}

# 4. SELLER thêm sản phẩm: 201
curl -i -X POST http://localhost:8080/api/products \
     -H "Authorization: Bearer <TOKEN_SELLER>" -H "Content-Type: application/json" \
     -d "{\"name\":\"Tai nghe Sony WF-
C700N\",\"price\":2290000,\"quantity\":20,\"categoryId\":5,\"imageUrl\":\"https://placehold.co/400x
```

```
300?text=WF-C700N\"}"

# 5. Lấy token CUSTOMER rồi thử xoá sản phẩm: 403
curl -i -X DELETE http://localhost:8080/api/products/1 -H "Authorization: Bearer <TOKEN_CUSTOMER>"

# 6. Token bị sửa 1 ký tự / hết hạn: 401
curl -i -X DELETE http://localhost:8080/api/products/1 -H "Authorization: Bearer abc.def.ghi"
```

Trong Postman: tab **Authorization → Type: Bearer Token** → dán token. Dán token vào https://jwt.io để xem payload `sub`, `roles`, `iat`, `exp`.

Lỗi thường gặp

Hiện tượng	**Nguyên nhân / cách sửa**
`release version 17 not supported` khi build bằng Maven	Maven đang chạy bằng JDK cũ hơn 17 → *Settings* → *Build Tools* → *Maven* → *Importing* → *JDK for importer* chọn **17**; kiểm tra `mvn -v` và `JAVA_HOME`
`invalid source release: 17` (IntelliJ báo khi build)	Project SDK / Language level chưa phải 17 → *File* → *Project Structure* → *Project* chọn **17**; kiểm tra `<java.version>17</java.version>` trong `pom.xml`
`Could not transfer artifact org.springframework.boot:...`	Máy chưa có Internet và kho `.m2` trống – xem mục 0.3
Lombok: `cannot find symbol getXxx()`	Bật *Enable annotation processing*; kiểm tra `pom.xml` có dependency `org.projectlombok:lombok`
`Access denied for user 'root'@'localhost'`	Sai mật khẩu MariaDB trong `application.properties` (và không để chú thích cùng dòng)
`Table 'eshop.xxx' doesn't exist`	Chưa chạy đủ `eShop.sql` hoặc sai tên DB trong URL
`LazyInitializationException` trong template	Đã đặt `spring.jpa.open-in-view=false` – để `true` hoặc dùng `JOIN FETCH`
Ảnh sản phẩm không hiện	Máy không có Internet (placehold.co) – dùng ảnh trong `static/images`
Vừa thêm starter security, mọi trang bắt đăng nhập với mật khẩu sinh ngẫu nhiên	Chưa tạo `SecurityConfig` / `CustomUserDetailsService`
POST form bị **403**	Thiếu CSRF token: dùng `th:action="@{...}"` thay vì `action="..."`

Hiện tượng	Nguyên nhân / cách sửa
Đăng xuất bằng `<a href="/logout">` không chạy	Logout phải là **form POST**
Đăng nhập đúng vẫn báo sai	Hash BCrypt trong DB bị cắt/sai; cột `password` < 60 ký tự
Có `ROLE_SELLER` mà vẫn 403	DB lưu `SELLER` (thiếu `ROLE_`), hoặc code viết `hasRole('ROLE_SELLER')`
`sec:authorize` không có tác dụng	Thiếu `thymeleaf-extras-springsecurity6`
`JwtAuthenticationFilter` chạy 2 lần	Đã đánh dấu `@Component` cho filter – bỏ đi, tạo bằng `new` trong `SecurityConfig`
`WeakKeyException: ... key byte array is 128 bits`	`app.jwt.secret` quá ngắn – cần ≥ 32 byte sau khi giải mã Base64
Đăng nhập xong vẫn là GUEST	Cookie bị chặn: kiểm tra `path("/")`; nếu đặt `.secure(true)` thì phải chạy HTTPS
Khoá tài khoản nhưng người đó vẫn dùng được	Filter lấy quyền từ token thay vì DB – bài này đã nạp lại từ DB mỗi request

Bài tập mở rộng

1. **Upload ảnh** sản phẩm bằng `MultipartFile` thay cho nhập URL, lưu vào thư mục `uploads/` và phục vụ qua `WebMvcConfigurer.addResourceHandlers`.
2. **Giỏ hàng lưu DB** (bảng `cart_item(user_id, product_id, quantity)`) để giữ giỏ khi đăng nhập trên thiết bị khác; gộp giỏ session vào DB khi đăng nhập.
3. **Chống bán vượt tồn kho khi nhiều người đặt cùng lúc**: thêm cột `version` + `@Version` (optimistic locking) cho `Product`, hoặc `@Lock(PESSIMISTIC_WRITE)`.
4. **Refresh token**: access token 15 phút + refresh token 7 ngày lưu DB; endpoint `/api/auth/refresh`; thu hồi khi đăng xuất.
5. **Tích hợp cổng thanh toán thật** ở chế độ sandbox (VNPay, MoMo): tạo URL thanh toán, xử lý `return_url` và kiểm chữ ký.
6. Gửi **email xác nhận đơn hàng** bằng `spring-boot-starter-mail`.
7. **Thống kê** cho SELLER: doanh thu theo ngày/tháng, top 5 sản phẩm bán chạy (JPQL `GROUP BY`), vẽ biểu đồ bằng Chart.js.
8. Viết **test** với `@WebMvcTest` + `@WithMockUser(roles = "SELLER")`, và `@DataJpaTest` cho `ProductRepository.search`.